

Video Game Sales Analysis Project

Project Overview

In this project, you'll analyze video game sales data to identify patterns that determine a game's success. Working as an analyst for the online store Ice, you'll use this information to help plan future advertising campaigns.

Environment Setup and Required Libraries

Hi there, in this project I will be analyzing the games sales data in order to determine what helpful information to help plan future advertising campaigns.

```
In [1]: import pandas as pd
import numpy as np
import math
import matplotlib.pyplot as plt
from scipy import stats as st
import seaborn as sns
# Import all required libraries
```

Step 1: Loading and Initial Data Exploration

First, let's load our dataset and examine its basic properties:

```
In [2]: games_sales_df = pd.read_csv('/datasets/games.csv')# Load the dataset
```

```
In [3]: display(games_sales_df.head())

display(games_sales_df.info())

display(games_sales_df.describe())
# Display basic information about the dataset
```

	Name	Platform	Year_of_Release	Genre	NA_sales	EU_sales	JP_sales	Other_sale
0	Wii Sports	Wii	2006.0	Sports	41.36	28.96	3.77	8.4
1	Super Mario Bros.	NES	1985.0	Platform	29.08	3.58	6.81	0.7
2	Mario Kart Wii	Wii	2008.0	Racing	15.68	12.76	3.79	3.2
3	Wii Sports Resort	Wii	2009.0	Sports	15.61	10.93	3.28	2.9
4	Pokemon Red/Pokemon Blue	GB	1996.0	Role-Playing	11.27	8.89	10.22	1.0



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16715 entries, 0 to 16714
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Name                   16713 non-null  object
1   Platform               16715 non-null  object
2   Year_of_Release        16446 non-null  float64
3   Genre                  16713 non-null  object
4   NA_sales               16715 non-null  float64
5   EU_sales               16715 non-null  float64
6   JP_sales               16715 non-null  float64
7   Other_sales            16715 non-null  float64
8   Critic_Score           8137 non-null   float64
9   User_Score             10014 non-null  object
10  Rating                 9949 non-null   object
dtypes: float64(6), object(5)
memory usage: 1.4+ MB
None
```

	Year_of_Release	NA_sales	EU_sales	JP_sales	Other_sales	Critic_Score
count	16446.000000	16715.000000	16715.000000	16715.000000	16715.000000	8137.000000
mean	2006.484616	0.263377	0.145060	0.077617	0.047342	68.967679
std	5.877050	0.813604	0.503339	0.308853	0.186731	13.938165
min	1980.000000	0.000000	0.000000	0.000000	0.000000	13.000000
25%	2003.000000	0.000000	0.000000	0.000000	0.000000	60.000000
50%	2007.000000	0.080000	0.020000	0.000000	0.010000	71.000000
75%	2010.000000	0.240000	0.110000	0.040000	0.030000	79.000000
max	2016.000000	41.360000	28.960000	10.220000	10.570000	98.000000



```
In [4]: games_sales_df.head()
```

```
Out[4]:
```

	Name	Platform	Year_of_Release	Genre	NA_sales	EU_sales	JP_sales	Other_sales
0	Wii Sports	Wii	2006.0	Sports	41.36	28.96	3.77	8.46
1	Super Mario Bros.	NES	1985.0	Platform	29.08	3.58	6.81	0.5
2	Mario Kart Wii	Wii	2008.0	Racing	15.68	12.76	3.79	3.28
3	Wii Sports Resort	Wii	2009.0	Sports	15.61	10.93	3.28	2.02
4	Pokemon Red/Pokemon Blue	GB	1996.0	Role-Playing	11.27	8.89	10.22	0.5

```
In [5]: duplicate = games_sales_df[games_sales_df.duplicated()]
print(duplicate)# Check for duplicate entries
```

Empty DataFrame

Columns: [Name, Platform, Year_of_Release, Genre, NA_sales, EU_sales, JP_sales, Other_sales, Critic_Score, User_Score, Rating]

Index: []

Key Questions to Answer:

- What's the total number of records in our dataset?
- What data types are present in each column?
- Are there any obvious issues with the data?
- Do we see any immediate patterns or anomalies?

We have in total 16714 number of records in the dataset. Also, the data types present in each column are float64 and object. I have found a good amount of missing data, however, I choose to not replace for any values or words so I could calculate the mean and the variance without any issues. I am able to see that overall North American countries buy more games than Europe and Japan.

Step 2: Data Preparation

2.1 Standardizing Column Names

```
In [6]: games_sales_df.columns = games_sales_df.columns.str.lower()# Convert column names to lowercase
```

```
In [7]: print(games_sales_df.columns)# Verify the changes
```

```
Index(['name', 'platform', 'year_of_release', 'genre', 'na_sales', 'eu_sales',  
      'jp_sales', 'other_sales', 'critic_score', 'user_score', 'rating'],  
      dtype='object')
```

2.2 Data Type Conversion

```
In [8]: games_sales_df.dtypes# Check current data types
```

```
Out[8]: name                object  
platform                object  
year_of_release        float64  
genre                  object  
na_sales               float64  
eu_sales               float64  
jp_sales               float64  
other_sales            float64  
critic_score           float64  
user_score             object  
rating                 object  
dtype: object
```

```
In [9]: games_sales_df['user_score'].value_counts()  
games_sales_df['user_score'] = games_sales_df['user_score'].replace('tbd', np.nan)  
print(games_sales_df['user_score'])  
  
games_sales_df['user_score'] = pd.to_numeric(games_sales_df['user_score'], errors='  
games_sales_df.info()# Make changes to data types if necessary  
# Describe the columns where the data types have been changed and why.
```

```

0      8
1     NaN
2     8.3
3      8
4     NaN
...
16710  NaN
16711  NaN
16712  NaN
16713  NaN
16714  NaN
Name: user_score, Length: 16715, dtype: object
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16715 entries, 0 to 16714
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   name                   16713 non-null  object
1   platform               16715 non-null  object
2   year_of_release        16446 non-null  float64
3   genre                  16713 non-null  object
4   na_sales                16715 non-null  float64
5   eu_sales                16715 non-null  float64
6   jp_sales                16715 non-null  float64
7   other_sales             16715 non-null  float64
8   critic_score            8137 non-null   float64
9   user_score              7590 non-null   float64
10  rating                  9949 non-null   object
dtypes: float64(7), object(4)
memory usage: 1.4+ MB

```

I have replace 'tbd' for NaN

I did replacement so when I calculate the statistical concepts to have a better understanding of the analysis, it will not give me any problem.

2.3 Handling Missing Values

```
In [10]: print(games_sales_df.head(20))
```

	name	platform	year_of_release	\
0	Wii Sports	Wii	2006.0	
1	Super Mario Bros.	NES	1985.0	
2	Mario Kart Wii	Wii	2008.0	
3	Wii Sports Resort	Wii	2009.0	
4	Pokemon Red/Pokemon Blue	GB	1996.0	
5	Tetris	GB	1989.0	
6	New Super Mario Bros.	DS	2006.0	
7	Wii Play	Wii	2006.0	
8	New Super Mario Bros. Wii	Wii	2009.0	
9	Duck Hunt	NES	1984.0	
10	Nintendogs	DS	2005.0	
11	Mario Kart DS	DS	2005.0	
12	Pokemon Gold/Pokemon Silver	GB	1999.0	
13	Wii Fit	Wii	2007.0	
14	Kinect Adventures!	X360	2010.0	
15	Wii Fit Plus	Wii	2009.0	
16	Grand Theft Auto V	PS3	2013.0	
17	Grand Theft Auto: San Andreas	PS2	2004.0	
18	Super Mario World	SNES	1990.0	
19	Brain Age: Train Your Brain in Minutes a Day	DS	2005.0	

	genre	na_sales	eu_sales	jp_sales	other_sales	critic_score	\
0	Sports	41.36	28.96	3.77	8.45	76.0	
1	Platform	29.08	3.58	6.81	0.77	NaN	
2	Racing	15.68	12.76	3.79	3.29	82.0	
3	Sports	15.61	10.93	3.28	2.95	80.0	
4	Role-Playing	11.27	8.89	10.22	1.00	NaN	
5	Puzzle	23.20	2.26	4.22	0.58	NaN	
6	Platform	11.28	9.14	6.50	2.88	89.0	
7	Misc	13.96	9.18	2.93	2.84	58.0	
8	Platform	14.44	6.94	4.70	2.24	87.0	
9	Shooter	26.93	0.63	0.28	0.47	NaN	
10	Simulation	9.05	10.95	1.93	2.74	NaN	
11	Racing	9.71	7.47	4.13	1.90	91.0	
12	Role-Playing	9.00	6.18	7.20	0.71	NaN	
13	Sports	8.92	8.03	3.60	2.15	80.0	
14	Misc	15.00	4.89	0.24	1.69	61.0	
15	Sports	9.01	8.49	2.53	1.77	80.0	
16	Action	7.02	9.09	0.98	3.96	97.0	
17	Action	9.43	0.40	0.41	10.57	95.0	
18	Platform	12.78	3.75	3.54	0.55	NaN	
19	Misc	4.74	9.20	4.16	2.04	77.0	

	user_score	rating
0	8.0	E
1	NaN	NaN
2	8.3	E
3	8.0	E
4	NaN	NaN
5	NaN	NaN
6	8.5	E
7	6.6	E
8	8.4	E
9	NaN	NaN
10	NaN	NaN

11	8.6	E
12	NaN	NaN
13	7.7	E
14	6.3	E
15	7.4	E
16	8.2	M
17	9.0	M
18	NaN	NaN
19	7.9	E

```
In [11]: percent_missing_values = games_sales_df.isnull().sum() *100 / len(games_sales_df)
print(percent_missing_values)# Calculate percentage of missing values
```

```
name          0.011965
platform      0.000000
year_of_release 1.609333
genre         0.011965
na_sales      0.000000
eu_sales      0.000000
jp_sales      0.000000
other_sales   0.000000
critic_score   51.319174
user_score    54.591684
rating        40.478612
dtype: float64
```

```
In [12]: #We can see that the majority of the missing values are from the columns: 'critic_s
# Analyze patterns in missing values
```

```
In [13]: games_sales_df['rating'] = games_sales_df['rating'].fillna('missing')
games_sales_df = games_sales_df.dropna(subset = ['name', 'year_of_release', 'genre']

display(games_sales_df['rating'])
display(games_sales_df[['name', 'year_of_release', 'genre']])

percent_missing_values = games_sales_df.isnull().sum() *100 / len(games_sales_df)
print(percent_missing_values)
# Handle missing values based on analysis
# Your code here to handle missing values according to your strategy
```

```
0          E
1      missing
2          E
3          E
4      missing
...
16710  missing
16711  missing
16712  missing
16713  missing
16714  missing
Name: rating, Length: 16444, dtype: object
```

	name	year_of_release	genre
0	Wii Sports	2006.0	Sports
1	Super Mario Bros.	1985.0	Platform
2	Mario Kart Wii	2008.0	Racing
3	Wii Sports Resort	2009.0	Sports
4	Pokemon Red/Pokemon Blue	1996.0	Role-Playing
...	...	...	...
16710	Samurai Warriors: Sanada Maru	2016.0	Action
16711	LMA Manager 2007	2006.0	Sports
16712	Haitaka no Psychedelica	2016.0	Adventure
16713	Spirits & Spells	2003.0	Platform
16714	Winning Post 8 2016	2016.0	Simulation

16444 rows × 3 columns

```

name          0.000000
platform      0.000000
year_of_release 0.000000
genre         0.000000
na_sales      0.000000
eu_sales      0.000000
jp_sales      0.000000
other_sales   0.000000
critic_score  51.453418
user_score    54.615665
rating        0.000000
dtype: float64

```

Why do you think the values are missing? Give possible reasons.

Explain why you filled in the missing values as you did or why you decided to leave them blank.

I replace the dropped NaN values for the columns that the percentage was very low, and replace the missing values for "missing" for the 'rating' column. However, I choose to not do anything with 'critic_score' and 'user_score' due to their high percentage of missing values, so I

could potentially calculate stats concepts to have a better analysis later on.

2.4 Calculate Total Sales

```
In [14]: games_sales_df['total_sales'] = games_sales_df[['na_sales', 'eu_sales', 'jp_sales'],
display(games_sales_df['total_sales'])
# Calculate total sales across all regions and put them in a different column
```

0	82.54
1	40.24
2	35.52
3	32.77
4	31.38
	...
16710	0.01
16711	0.01
16712	0.01
16713	0.01
16714	0.01

Name: total_sales, Length: 16444, dtype: float64

I have successfully calculated total sales, creating a new column for it."

Step 3: Analyzing Video Game Sales Data

3.1 Temporal Analysis of Game Releases

Let's first examine the distribution of game releases across different years to understand our data's coverage and significance:

```
In [15]: game_release_by_year = games_sales_df[['name', 'year_of_release']]
display(game_release_by_year) # Create a DataFrame with game releases by year
```

	name	year_of_release
0	Wii Sports	2006.0
1	Super Mario Bros.	1985.0
2	Mario Kart Wii	2008.0
3	Wii Sports Resort	2009.0
4	Pokemon Red/Pokemon Blue	1996.0
...	...	...
16710	Samurai Warriors: Sanada Maru	2016.0
16711	LMA Manager 2007	2006.0
16712	Haitaka no Psychedelica	2016.0
16713	Spirits & Spells	2003.0
16714	Winning Post 8 2016	2016.0

16444 rows × 2 columns

Data frame with the game releases by year created.

```
In [16]: #First, let's count how many games were release in each year
games_per_year = games_sales_df['year_of_release'].value_counts().sort_index()
print(games_per_year.head())

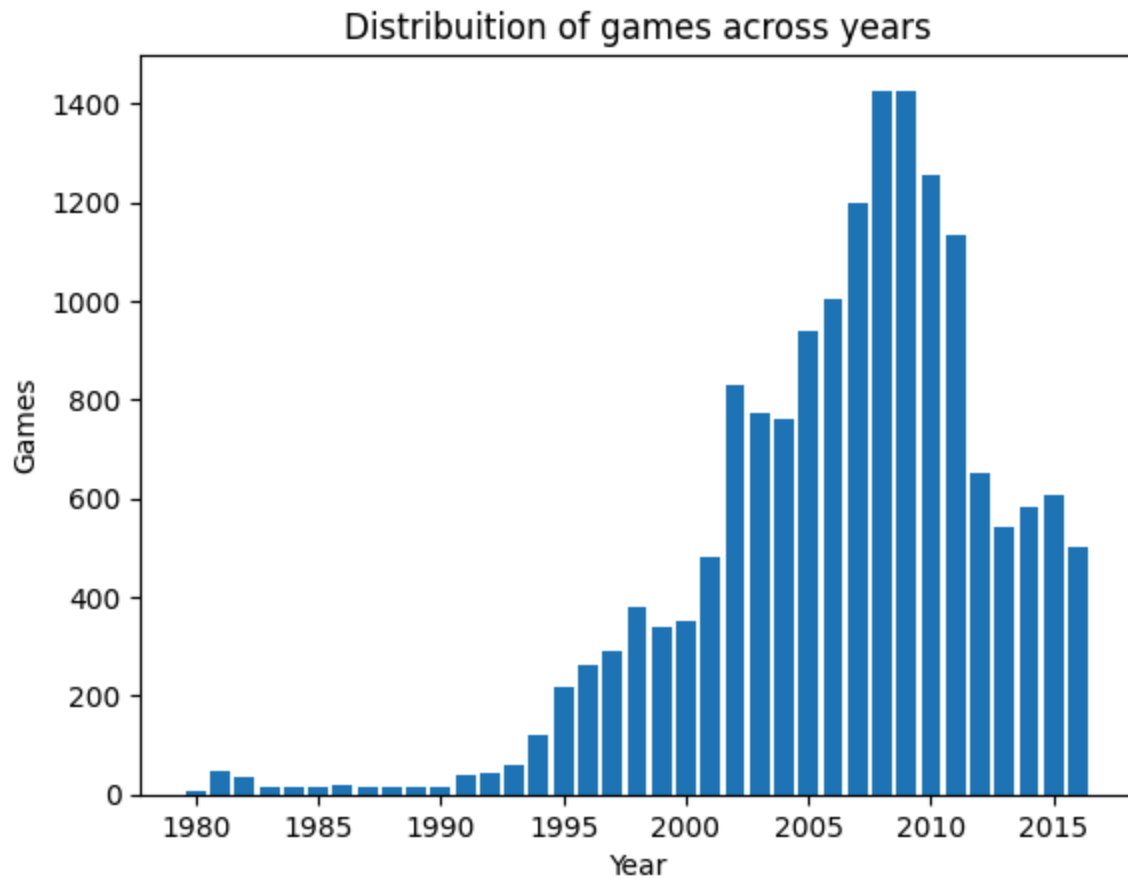
#Now, I will plot into the graphic

x = ['Years']
y = ['Games released per year']

plt.bar(games_per_year.index, games_per_year.values)
plt.title("Distribution of games across years")
plt.xlabel("Year")
plt.ylabel("Games")
plt.show()

# Visualize the distribution of games across years
```

```
1980.0    9
1981.0   46
1982.0   36
1983.0   17
1984.0   14
Name: year_of_release, dtype: int64
```



```
In [17]: # Display summary statistics for each year
print("Summary Statistics:")
print(games_sales_df.groupby('year_of_release').describe())
```

Summary Statistics:

	na_sales \						
	count	mean	std	min	25%	50%	75%
year_of_release							
1980.0	9.0	1.176667	1.282624	0.22	0.3200	0.720	1.0700
1981.0	46.0	0.726087	0.730885	0.12	0.2925	0.435	0.8925
1982.0	36.0	0.747778	1.172248	0.13	0.3150	0.500	0.7225
1983.0	17.0	0.456471	0.278903	0.01	0.3100	0.430	0.5100
1984.0	14.0	2.377143	7.089785	0.00	0.0450	0.345	0.7700
1985.0	14.0	2.409286	7.689309	0.00	0.0000	0.285	0.6025
1986.0	21.0	0.595238	0.881542	0.00	0.0000	0.320	0.7100
1987.0	16.0	0.528750	0.653094	0.00	0.1350	0.370	0.4875
1988.0	15.0	1.591333	2.630154	0.00	0.1000	0.590	1.3700
1989.0	17.0	2.655882	5.893720	0.00	0.1400	0.660	0.9600
1990.0	16.0	1.591250	3.087251	0.00	0.2300	0.675	1.7325
1991.0	41.0	0.311220	0.643619	0.00	0.0000	0.000	0.4100
1992.0	43.0	0.788140	1.359452	0.00	0.0000	0.020	1.1000
1993.0	60.0	0.252000	0.831529	0.00	0.0000	0.000	0.0525
1994.0	121.0	0.232727	0.611660	0.00	0.0000	0.000	0.0200
1995.0	219.0	0.113379	0.347959	0.00	0.0000	0.000	0.0400
1996.0	263.0	0.329886	1.007959	0.00	0.0000	0.040	0.1650
1997.0	289.0	0.327855	0.711767	0.00	0.0000	0.070	0.2900
1998.0	379.0	0.338681	0.646139	0.00	0.0100	0.080	0.3600
1999.0	338.0	0.372959	0.788408	0.00	0.0200	0.130	0.3575
2000.0	350.0	0.270000	0.457221	0.00	0.0200	0.100	0.2875
2001.0	482.0	0.360954	0.734771	0.00	0.0325	0.115	0.3300
2002.0	829.0	0.260784	0.561713	0.00	0.0300	0.090	0.2300
2003.0	775.0	0.249819	0.402497	0.00	0.0400	0.110	0.2600
2004.0	762.0	0.292008	0.611504	0.00	0.0400	0.110	0.3275
2005.0	939.0	0.257881	0.590938	0.00	0.0400	0.100	0.2600
2006.0	1006.0	0.260567	1.478435	0.00	0.0000	0.050	0.1900
2007.0	1197.0	0.258889	0.608283	0.00	0.0000	0.090	0.2700
2008.0	1427.0	0.244352	0.631926	0.00	0.0100	0.090	0.2500
2009.0	1426.0	0.235309	0.760085	0.00	0.0000	0.090	0.2100
2010.0	1255.0	0.239562	0.732948	0.00	0.0000	0.080	0.1900
2011.0	1136.0	0.210202	0.580080	0.00	0.0000	0.060	0.1700
2012.0	653.0	0.234701	0.626764	0.00	0.0000	0.050	0.2100
2013.0	544.0	0.282445	0.777533	0.00	0.0000	0.045	0.2400
2014.0	581.0	0.227659	0.515008	0.00	0.0000	0.050	0.2000
2015.0	606.0	0.176337	0.487092	0.00	0.0000	0.020	0.1300
2016.0	502.0	0.089502	0.258167	0.00	0.0000	0.010	0.0500

	eu_sales			...	user_score	total_sales \	
	max	count	mean	...	75%	max	count
year_of_release							
1980.0	4.00	9.0	0.074444	...	NaN	NaN	9.0
1981.0	4.21	46.0	0.042609	...	NaN	NaN	46.0
1982.0	7.28	36.0	0.045833	...	NaN	NaN	36.0
1983.0	1.22	17.0	0.047059	...	NaN	NaN	17.0
1984.0	26.93	14.0	0.150000	...	NaN	NaN	14.0
1985.0	29.08	14.0	0.338571	...	5.80	5.8	14.0
1986.0	3.74	21.0	0.135238	...	NaN	NaN	21.0
1987.0	2.19	16.0	0.088125	...	NaN	NaN	16.0
1988.0	9.54	15.0	0.439333	...	2.20	2.2	15.0
1989.0	23.20	17.0	0.496471	...	NaN	NaN	17.0
1990.0	12.78	16.0	0.476875	...	NaN	NaN	16.0

1991.0	3.03	41.0	0.096341	...	NaN	NaN	41.0
1992.0	6.16	43.0	0.272326	...	8.20	8.2	43.0
1993.0	5.99	60.0	0.077500	...	NaN	NaN	60.0
1994.0	4.36	121.0	0.122975	...	6.30	6.3	121.0
1995.0	2.26	219.0	0.068037	...	NaN	NaN	219.0
1996.0	11.27	263.0	0.179696	...	8.75	9.0	263.0
1997.0	5.80	289.0	0.167197	...	9.00	9.4	289.0
1998.0	5.89	379.0	0.176517	...	9.05	9.4	379.0
1999.0	9.00	338.0	0.185414	...	9.00	9.4	338.0
2000.0	3.05	350.0	0.150771	...	8.45	9.4	350.0
2001.0	6.99	482.0	0.196867	...	8.60	9.4	482.0
2002.0	8.41	829.0	0.132388	...	8.50	9.5	829.0
2003.0	4.12	775.0	0.133948	...	8.70	9.6	775.0
2004.0	9.43	762.0	0.140787	...	8.60	9.5	762.0
2005.0	9.71	939.0	0.128978	...	8.50	9.7	939.0
2006.0	41.36	1006.0	0.127127	...	8.20	9.3	1006.0
2007.0	8.92	1197.0	0.131846	...	7.90	9.5	1197.0
2008.0	15.68	1427.0	0.126938	...	7.90	9.2	1427.0
2009.0	15.61	1426.0	0.131795	...	8.10	9.3	1426.0
2010.0	15.00	1255.0	0.136590	...	7.90	9.1	1255.0
2011.0	9.04	1136.0	0.143460	...	7.70	8.8	1136.0
2012.0	8.25	653.0	0.175482	...	7.90	9.2	653.0
2013.0	9.66	544.0	0.223438	...	7.70	9.2	544.0
2014.0	4.35	581.0	0.211256	...	7.70	9.1	581.0
2015.0	6.03	606.0	0.159604	...	7.80	9.3	606.0
2016.0	2.98	502.0	0.102032	...	7.90	8.8	502.0

	mean	std	min	25%	50%	75%	max
year_of_release							
1980.0	1.264444	1.383547	0.23	0.3400	0.770	1.1500	4.31
1981.0	0.775652	0.783019	0.13	0.3125	0.465	0.9525	4.50
1982.0	0.802222	1.257910	0.14	0.3350	0.540	0.7800	7.81
1983.0	0.988235	0.838825	0.16	0.4000	0.770	1.1300	3.20
1984.0	3.596429	7.200795	0.27	1.1300	1.485	2.0900	28.31
1985.0	3.853571	10.510376	0.02	0.4650	1.015	1.4550	40.24
1986.0	1.765714	1.328001	0.30	1.1500	1.360	2.2000	6.50
1987.0	1.356250	1.225174	0.07	0.4550	0.885	2.2125	4.38
1988.0	3.147333	4.414414	0.03	0.9700	1.510	3.1450	17.28
1989.0	4.320588	7.895161	0.06	1.1000	1.610	2.1200	30.26
1990.0	3.085625	4.827705	0.30	1.1650	1.520	2.9100	20.62
1991.0	0.786098	1.008699	0.05	0.2300	0.380	1.0000	4.61
1992.0	1.771163	2.469928	0.02	0.1600	0.790	2.3200	11.18
1993.0	0.766500	1.440388	0.01	0.1100	0.355	0.9900	10.55
1994.0	0.654793	1.184502	0.02	0.0800	0.270	0.5900	9.30
1995.0	0.402374	0.760314	0.01	0.0700	0.140	0.3500	5.15
1996.0	0.757224	2.304625	0.01	0.1000	0.210	0.5500	31.38
1997.0	0.695744	1.289182	0.01	0.1300	0.280	0.7100	10.95
1998.0	0.676280	1.250623	0.02	0.0900	0.260	0.7150	14.64
1999.0	0.742929	1.681434	0.01	0.1300	0.305	0.6575	23.09
2000.0	0.576171	0.882493	0.01	0.1100	0.250	0.5975	6.39
2001.0	0.687676	1.358480	0.01	0.1025	0.250	0.6000	14.97
2002.0	0.476441	1.083014	0.01	0.0700	0.170	0.4400	16.15
2003.0	0.461342	0.758530	0.01	0.0800	0.190	0.4900	7.20
2004.0	0.549449	1.259074	0.01	0.0800	0.210	0.5400	20.81
2005.0	0.487561	1.557763	0.01	0.0750	0.170	0.4300	24.67

2006.0	0.514622	3.035907	0.01	0.0400	0.110	0.3375	82.54
2007.0	0.505221	1.215361	0.01	0.0500	0.150	0.4700	22.70
2008.0	0.470568	1.290517	0.00	0.0600	0.160	0.4800	35.52
2009.0	0.462006	1.554459	0.01	0.0600	0.150	0.4000	32.77
2010.0	0.470223	1.287804	0.01	0.0500	0.140	0.3900	21.82
2011.0	0.446813	1.140302	0.01	0.0600	0.130	0.3500	14.73
2012.0	0.544931	1.295684	0.01	0.0400	0.170	0.5100	13.79
2013.0	0.664044	1.662407	0.01	0.0500	0.190	0.5925	21.05
2014.0	0.570620	1.232507	0.01	0.0600	0.160	0.5300	12.62
2015.0	0.442211	1.099421	0.01	0.0300	0.090	0.3575	14.63
2016.0	0.258845	0.698041	0.01	0.0200	0.060	0.2000	7.60

[37 rows x 56 columns]

We can see the 'decade' the had the most games released was between 2002 and 2012. With that statistics, we are able to visualize that.

Questions to Consider:

- Which years show significant numbers of game releases?
- Are there any notable trends or patterns in the number of releases?
- Is there enough recent data to make predictions for 2017?

2008 and 2009 are the years that had the most game releases, totalizing 1427 and 1426, respectively.

3.2 Platform Sales Analysis Over Time

Now let's analyze how sales vary across platforms and years:

```
In [18]: total_sales_by_platform_per_year = games_sales_df[['platform', 'year_of_release']].
display(total_sales_by_platform_per_year)
# Calculate total sales by platform and year
```

```
0      2006.0
1      1985.0
2      2008.0
3      2009.0
4      1996.0
...
16710   2016.0
16711   2006.0
16712   2016.0
16713   2003.0
16714   2016.0
Length: 16444, dtype: float64
```

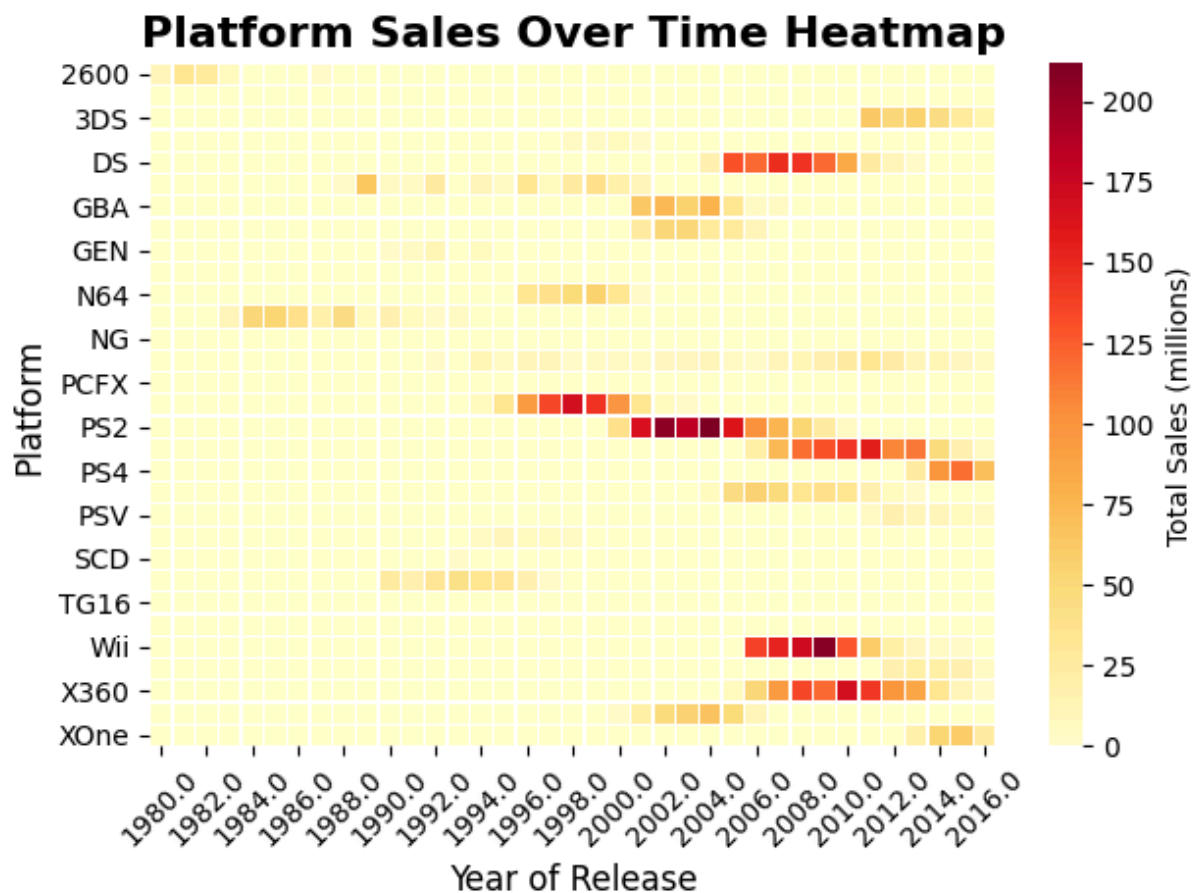
```
In [19]: total_sales_by_platform_by_year = (
games_sales_df.groupby(['platform', 'year_of_release'])['total_sales']
```

```

        .sum()
        .unstack(fill_value=0)
    )
    sns.heatmap(
        total_sales_by_platform_by_year,
        cmap='YlOrRd',
        cbar_kws={'label': 'Total Sales (millions)'},
        linewidths=0.1,
        fmt='.1f'
    )

    plt.title('Platform Sales Over Time Heatmap', fontsize=16, fontweight='bold')
    plt.xlabel('Year of Release', fontsize=12)
    plt.ylabel('Platform', fontsize=12)
    plt.xticks(rotation=45)
    plt.yticks(rotation=0)
    plt.tight_layout()
    plt.show()# Create a heatmap of platform sales over time

```



Identify platforms with declining sales

With the heatmap we are able to identify some trends on different platform. A couple platform have decreased the sales over the years, such as: '2600', 'N64', 'NG', etc.

Questions to Consider:

- Which platforms show consistent sales over time?
- Can you identify platforms that have disappeared from the market?
- What's the typical lifecycle of a gaming platform?

As shown on the heatmap, 'Playstation,' 'Wii,' 'Xbox' are the platforms with the most consistent sales over the time. Some of the platforms have disappeared throughout the years, such as: '2600,' 'GEN,' 'SCD,' 'TG16'. The lifecycle of a gaming platform is around 5 to 10 years.

3.3 Determining Relevant Time Period

Based on your analysis above, determine the appropriate time period for predicting 2017 sales:

```
In [20]: relevant_years = [2012, 2013, 2014, 2015, 2016]
games_relevant_years = games_sales_df[games_sales_df['year_of_release'].isin(relevant_years)]

print(games_relevant_years) # Your code here to filter the dataset to relevant years
# Example:
# relevant_years = [XXXX, XXXX, XXXX] # Replace with your chosen years
# df_relevant = df[df['year_of_release'].isin(relevant_years)]

# Justify your choice with data
```


	name	platform	year_of_release	genre	\
16	Grand Theft Auto V	PS3	2013.0	Action	
23	Grand Theft Auto V	X360	2013.0	Action	
31	Call of Duty: Black Ops 3	PS4	2015.0	Shooter	
33	Pokemon X/Pokemon Y	3DS	2013.0	Role-Playing	
34	Call of Duty: Black Ops II	PS3	2012.0	Shooter	
...	...	...	...	...	
16703	Strawberry Nauts	PSV	2016.0	Adventure	
16707	Aiyoku no Eustia	PSV	2014.0	Misc	
16710	Samurai Warriors: Sanada Maru	PS3	2016.0	Action	
16712	Haitaka no Psychedelica	PSV	2016.0	Adventure	
16714	Winning Post 8 2016	PSV	2016.0	Simulation	

	na_sales	eu_sales	jp_sales	other_sales	critic_score	user_score	\
16	7.02	9.09	0.98	3.96	97.0	8.2	
23	9.66	5.14	0.06	1.41	97.0	8.1	
31	6.03	5.86	0.36	2.38	NaN	NaN	
33	5.28	4.19	4.35	0.78	NaN	NaN	
34	4.99	5.73	0.65	2.42	83.0	5.3	
...	...	...	...	...	...	...	
16703	0.00	0.00	0.01	0.00	NaN	NaN	
16707	0.00	0.00	0.01	0.00	NaN	NaN	
16710	0.00	0.00	0.01	0.00	NaN	NaN	
16712	0.00	0.00	0.01	0.00	NaN	NaN	
16714	0.00	0.00	0.01	0.00	NaN	NaN	

	rating	total_sales
16	M	21.05
23	M	16.27
31	missing	14.63
33	missing	14.60
34	M	13.79
...	...	...
16703	missing	0.01
16707	missing	0.01
16710	missing	0.01
16712	missing	0.01
16714	missing	0.01

[2886 rows x 12 columns]

Document Your Decision:

- What years did you select and why?
- How does this period reflect current market conditions?
- What factors influenced your decision?

I have chosen these years because they the most recent year, sot it will tell us what could potentially happen in 2017. Nowadays, PlayStation, Wii, and Xbox are some of the most popular video game platforms on the market. Although they are not the highest sales in comparison with other years, the years analyzed contributed to them to show us they've

continued to be as popular as they were when they came out. The millions of dollars in total sales during these years in comparison with the other platforms.

3.4 Platform Performance Analysis

Using your selected time period, let's analyze platform performance:

```
In [21]: platform_total_sales = games_relevant_years.groupby('platform')['total_sales'].sum()
print("Top 10 Platforms by Totals Sales:")
print(platform_total_sales.head(10))
# Analyze platform sales trends
# Sort platforms by total sales
```

Top 10 Platforms by Totals Sales:

platform	
PS4	314.14
PS3	288.79
X360	236.54
3DS	194.61
XOne	159.32
WiiU	82.19
PC	62.65
PSV	49.18
Wii	35.37
DS	12.55

Name: total_sales, dtype: float64

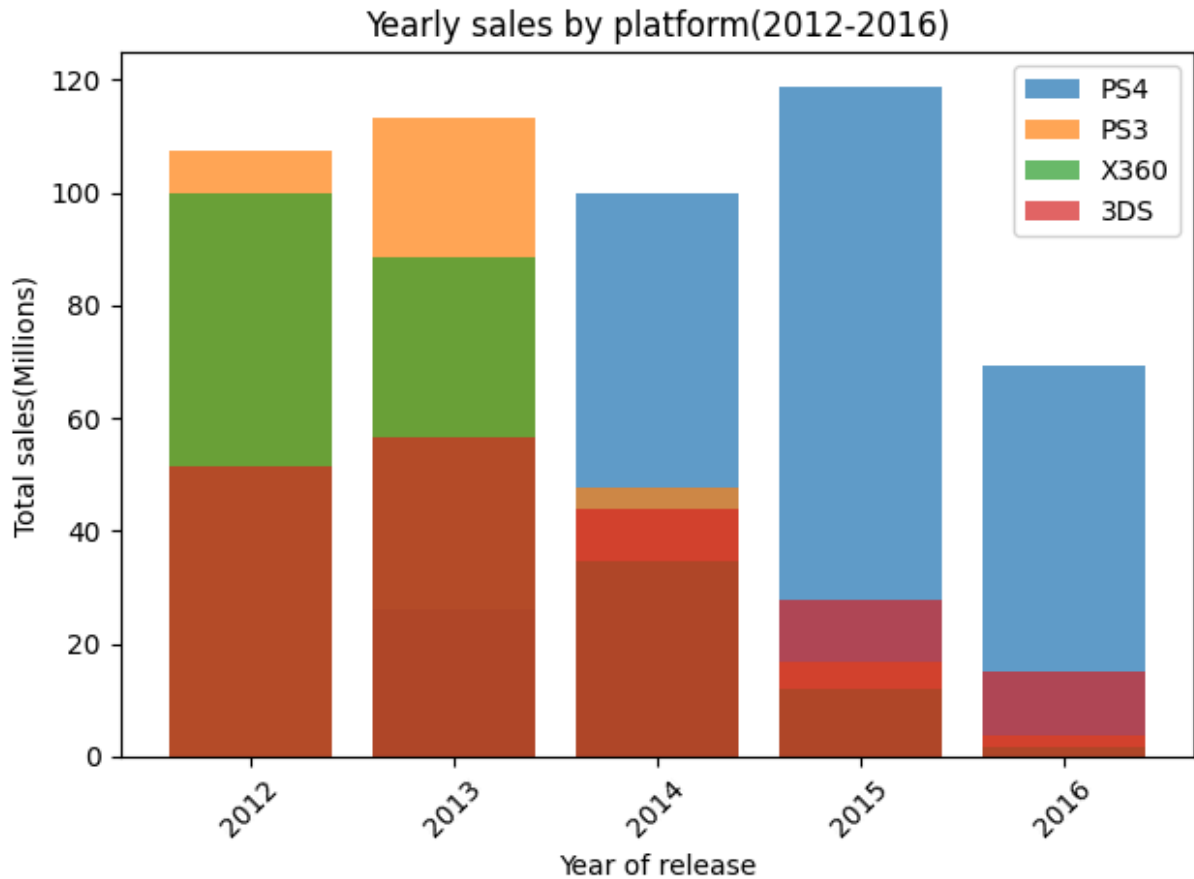
During the chosen time period, 'PS4' is the most sold platform.

```
In [22]: # Visualize top platforms
platform_yearly_sales = games_relevant_years.groupby(['platform', 'year_of_release'])
top_platforms = ['PS4', 'PS3', 'X360', '3DS']

platform_colors = {
    'PS4': 'tab:blue',
    'PS3': 'tab:orange',
    'X360': 'tab:green',
    '3DS': 'tab:red'
}

for platform in top_platforms:
    platform_data = platform_yearly_sales[platform]
    plt.bar(platform_data.index, platform_data.values, alpha=0.7, label=platform, c=platform_colors[platform])
    plt.legend()
plt.xlabel('Year of release')
plt.ylabel('Total sales(Millions)')
plt.title('Yearly sales by platform(2012-2016)')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

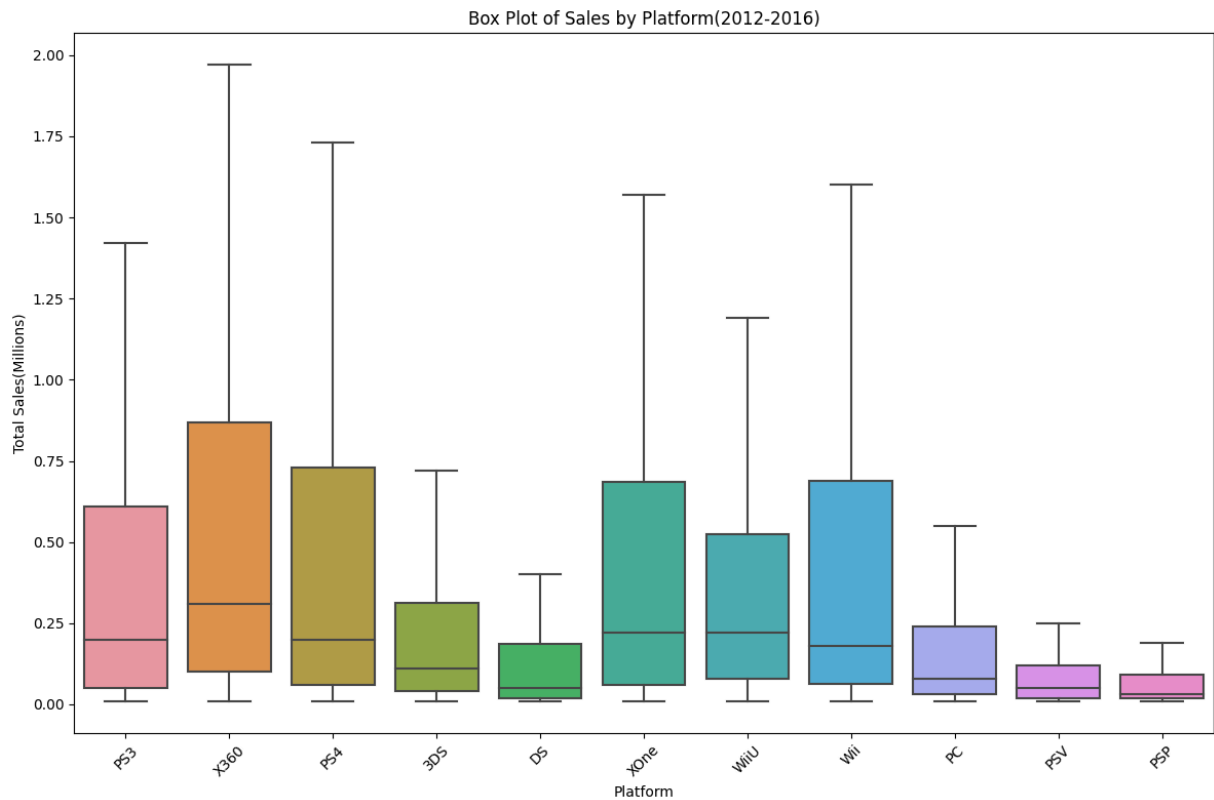
```
# Calculate year-over-year growth for each platform
# Your code here to calculate and visualize platform growth rates
```



3.5 Sales Distribution Analysis

Let's examine the distribution of sales across platforms:

```
In [23]: plt.figure(figsize=(12,8))
sns.boxplot(data=games_relevant_years, x='platform', y='total_sales', showfliers=False)
plt.xlabel("Platform")
plt.ylabel("Total Sales(Millions)")
plt.title("Box Plot of Sales by Platform(2012-2016)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()# Create box plot of sales by platform
```



```
In [24]: platform_stats = games_relevant_years.groupby('platform')['total_sales'].describe()
print("Platforms Statistics:")
print(platform_stats)
# Calculate detailed statistics for each platform
```

Platforms Statistics:

	count	mean	std	min	25%	50%	75%	max
platform								
3DS	396.0	0.491439	1.387161	0.01	0.0400	0.11	0.3125	14.60
DS	31.0	0.404839	1.439245	0.01	0.0200	0.05	0.1850	8.06
PC	250.0	0.250600	0.494640	0.01	0.0300	0.08	0.2400	5.14
PS3	493.0	0.585781	1.462855	0.01	0.0500	0.20	0.6100	21.05
PS4	392.0	0.801378	1.609456	0.01	0.0600	0.20	0.7300	14.63
PSP	173.0	0.064682	0.071296	0.01	0.0200	0.03	0.0900	0.36
PSV	411.0	0.119659	0.203011	0.01	0.0200	0.05	0.1200	1.96
Wii	54.0	0.655000	1.165447	0.01	0.0625	0.18	0.6875	6.76
WiiU	147.0	0.559116	1.058836	0.01	0.0800	0.22	0.5250	7.09
X360	292.0	0.810068	1.712069	0.01	0.1000	0.31	0.8700	16.27
XOne	247.0	0.645020	1.036139	0.01	0.0600	0.22	0.6850	7.39

3.6 Review Score Impact Analysis

Select a popular platform and analyze how reviews affect sales:

```
In [25]: ps4_info = games_relevant_years[games_relevant_years['platform']=='PS4']
print(ps4_info)
# Choose a popular platform based on your previous analysis
```

	name	platform	year_of_release	genre	\
31	Call of Duty: Black Ops 3	PS4	2015.0	Shooter	
42	Grand Theft Auto V	PS4	2014.0	Action	
77	FIFA 16	PS4	2015.0	Sports	
87	Star Wars Battlefront (2015)	PS4	2015.0	Shooter	
92	Call of Duty: Advanced Warfare	PS4	2014.0	Shooter	
...	...	...	...	...	
16500	Root Letter	PS4	2016.0	Adventure	
16503	Shin Hayarigami 2	PS4	2016.0	Adventure	
16526	Dungeons 2	PS4	2016.0	Role-Playing	
16530	Carmageddon: Max Damage	PS4	2016.0	Action	
16585	Farming 2017 - The Simulation	PS4	2016.0	Simulation	

	na_sales	eu_sales	jp_sales	other_sales	critic_score	user_score	\
31	6.03	5.86	0.36	2.38	NaN	NaN	
42	3.96	6.31	0.38	1.97	97.0	8.3	
77	1.12	6.12	0.06	1.28	82.0	4.3	
87	2.99	3.49	0.22	1.28	NaN	NaN	
92	2.81	3.48	0.14	1.23	83.0	5.7	
...	...	...	...	...	...	...	
16500	0.00	0.00	0.01	0.00	69.0	7.5	
16503	0.00	0.00	0.01	0.00	NaN	NaN	
16526	0.01	0.00	0.00	0.00	61.0	7.9	
16530	0.01	0.00	0.00	0.00	51.0	5.5	
16585	0.00	0.01	0.00	0.00	NaN	NaN	

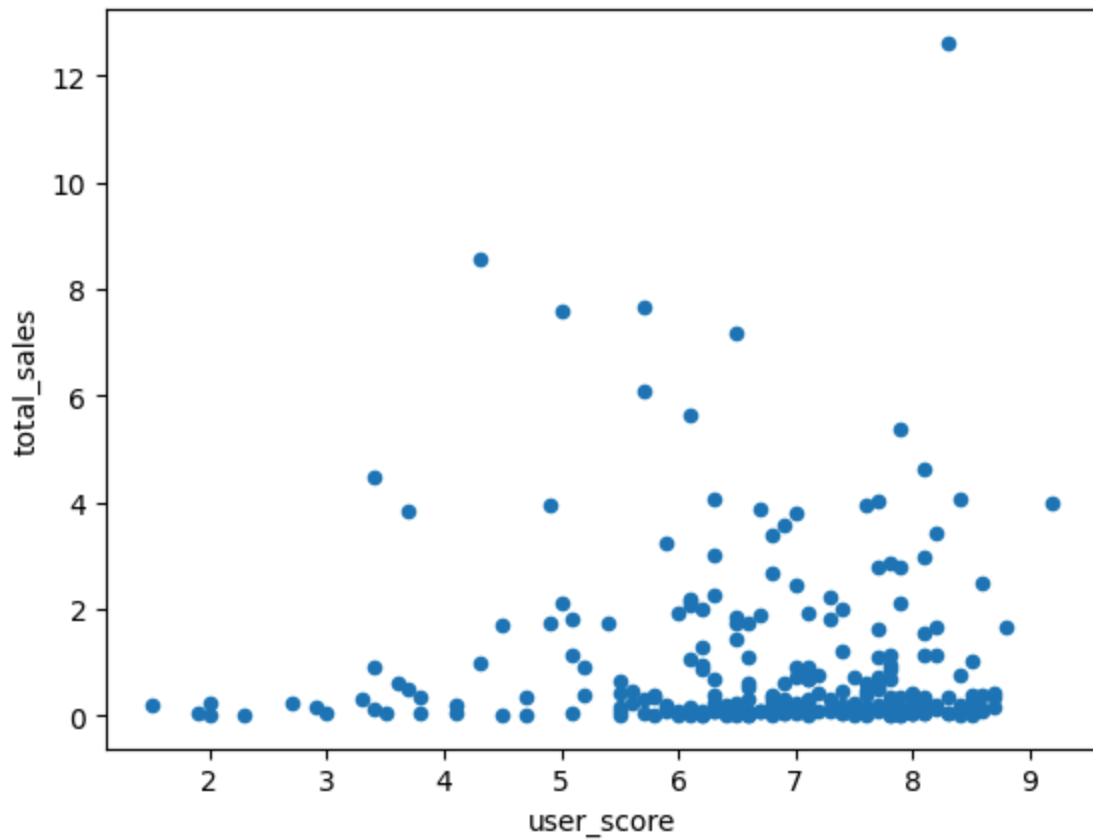
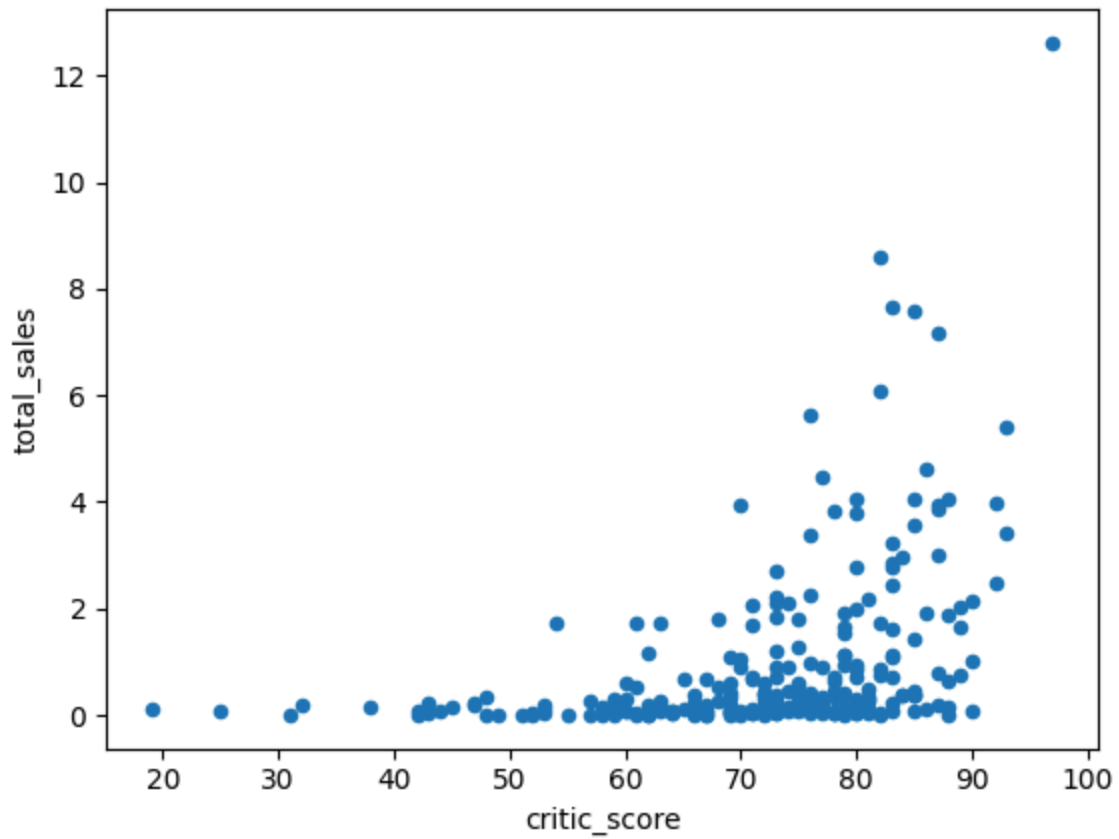
	rating	total_sales
31	missing	14.63
42	M	12.62
77	E	8.58
87	missing	7.98
92	M	7.66
...	...	...
16500	missing	0.01
16503	missing	0.01
16526	T	0.01
16530	M	0.01
16585	missing	0.01

[392 rows x 12 columns]

I have choose the platform PS4, for it is the most popular one according to the sales.

```
In [26]: ps4_info.plot(kind='scatter', x='critic_score', y='total_sales')
ps4_info.plot(kind='scatter', x='user_score', y='total_sales')# Create scatter plot
```

```
Out[26]: <AxesSubplot:xlabel='user_score', ylabel='total_sales'>
```



```
In [27]: ps4_info[['critic_score', 'user_score', 'total_sales']].corr()['total_sales']
```

```
Out[27]: critic_score    0.406568
user_score    -0.031957
total_sales    1.000000
Name: total_sales, dtype: float64
```

I have successfully plot the scatter graphic and shown the correlation between the columns.

3.7 Cross-Platform Comparison

Compare sales performance of games across different platforms:

```
In [28]: #Amount of platforms each game appears on
games_platform_count = games_relevant_years.groupby('name')['platform'].nunique()

#Finding games that appear on multiple platforms(more than 1)
multi_platform_games = games_platform_count[games_platform_count > 1]

print(f"Number of games released on multiple platforms: {len(multi_platform_games)}")
print("Top 10 games with most platform releases:")
print(multi_platform_games.sort_values(ascending=False).head(10))

# Find games released on multiple platforms
```

Number of games released on multiple platforms: 596

Top 10 games with most platform releases:

```
name
LEGO Marvel Super Heroes    9
FIFA 14                     9
The LEGO Movie Videogame    8
Angry Birds Star Wars       8
FIFA 15                     8
FIFA Soccer 13              8
LEGO The Hobbit             8
LEGO Jurassic World         8
Lego Batman 3: Beyond Gotham 8
LEGO The Lord of the Rings   7
Name: platform, dtype: int64
```

On the cell above, I have found the games released with more than one platform.

```
In [29]: # Find games released on multiple platforms

multi_platform_games = games_relevant_years.groupby('name')['platform'].nunique()

multi_platform_games = multi_platform_games[multi_platform_games > 1].index
#index= Ensuring in only has the ones more that one platform for the games
```

```
games_cross_platform_df = games_relevant_years[games_relevant_years['name'].isin(mu

games_cross_platform_df.head(10)
```

Out[29]:

	name	platform	year_of_release	genre	na_sales	eu_sales	jp_sales	other_sales
16	Grand Theft Auto V	PS3	2013.0	Action	7.02	9.09	0.98	3.96
23	Grand Theft Auto V	X360	2013.0	Action	9.66	5.14	0.06	1.41
31	Call of Duty: Black Ops 3	PS4	2015.0	Shooter	6.03	5.86	0.36	2.38
34	Call of Duty: Black Ops II	PS3	2012.0	Shooter	4.99	5.73	0.65	2.42
35	Call of Duty: Black Ops II	X360	2012.0	Shooter	8.25	4.24	0.07	1.12
42	Grand Theft Auto V	PS4	2014.0	Action	3.96	6.31	0.38	1.97
60	Call of Duty: Ghosts	X360	2013.0	Shooter	6.73	2.56	0.04	0.91
69	Call of Duty: Ghosts	PS3	2013.0	Shooter	4.10	3.63	0.38	1.25
72	Minecraft	X360	2013.0	Misc	5.70	2.65	0.02	0.81
77	FIFA 16	PS4	2015.0	Sports	1.12	6.12	0.06	1.28



On this cell, I was able to create a data frame that shows the games release in more than one platform

In the cell above, there was a calculation to find all the games released on multiple platforms in the time period chosen by me(2012-2016)


```

In [30]: #Fist, I will need to pick a popular multi-platform game to analyze
games_example = "FIFA 16"
fifa16_sales = games_cross_platform_df[games_cross_platform_df['name'] == games_example]
print(f"Sales comparison for {games_example}:")
print(fifa16_sales[['platform', 'total_sales']].sort_values('total_sales', ascending=False))

#Visualizing cross-platform performance for multi-platform games
multi_games = ['FIFA 16', 'Call of Duty: Black Ops II', 'Minecraft']

plt.figure(figsize=(12,8))
for game in multi_games:
    game_data = games_cross_platform_df[games_cross_platform_df['name'] == game]
    plt.bar(game_data['platform'], game_data['total_sales'], alpha=0.7, label=game)

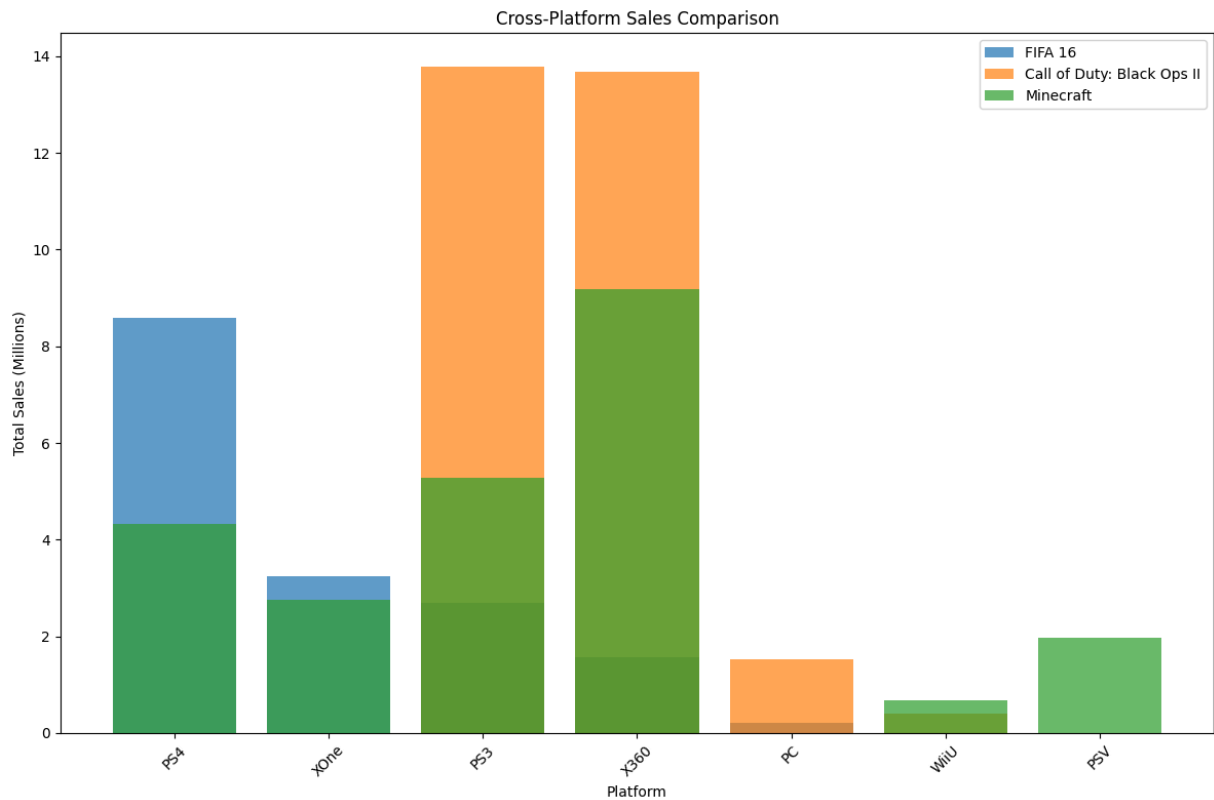
plt.xlabel('Platform')
plt.ylabel('Total Sales (Millions)')
plt.title('Cross-Platform Sales Comparison')
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Compare sales across platforms for these games
# Your code here to analyze and visualize cross-platform performance

```

Sales comparison for FIFA 16:

	platform	total_sales
77	PS4	8.58
417	XOne	3.25
552	PS3	2.70
1182	X360	1.57
7575	PC	0.20



Analyzing the sales for the game I picked (FIFA 16), we are able to see the differences of units sales per platform. Since the game came out in 2015, the newest platforms had the most sales, such as PS4 and XOne. People who had the oldest platforms also bought it, but in a small quantity. The plotting shows the cross-platform, analysing three games. We can see that, depending on the year the games come out, will directly influence on the percentage of sales for the platforms. For instance, 'FIFA16' came out in 2015, resulting in more sales for the latest platforms. On the other hand, 'Call of Duty' came out in 2012 where platforms like PS4 and XOne had not come out back then so the sales are less evident for these newest platforms.

3.8 Genre Analysis

Finally, let's examine the distribution of games by genre:

```
In [31]: print(games_relevant_years[['name', 'genre']].head(15))# Analyze genre performance
```

	name	genre
16	Grand Theft Auto V	Action
23	Grand Theft Auto V	Action
31	Call of Duty: Black Ops 3	Shooter
33	Pokemon X/Pokemon Y	Role-Playing
34	Call of Duty: Black Ops II	Shooter
35	Call of Duty: Black Ops II	Shooter
42	Grand Theft Auto V	Action
47	Pokemon Omega Ruby/Pokemon Alpha Sapphire	Role-Playing
60	Call of Duty: Ghosts	Shooter
62	New Super Mario Bros. 2	Platform
66	Halo 4	Shooter
69	Call of Duty: Ghosts	Shooter
72	Minecraft	Misc
73	Animal Crossing: New Leaf	Simulation
77	FIFA 16	Sports

```
In [32]: genre_sales = games_relevant_years.groupby('genre')['total_sales'].sum().sort_value
print(genre_sales)# Sort genres by total sales
```

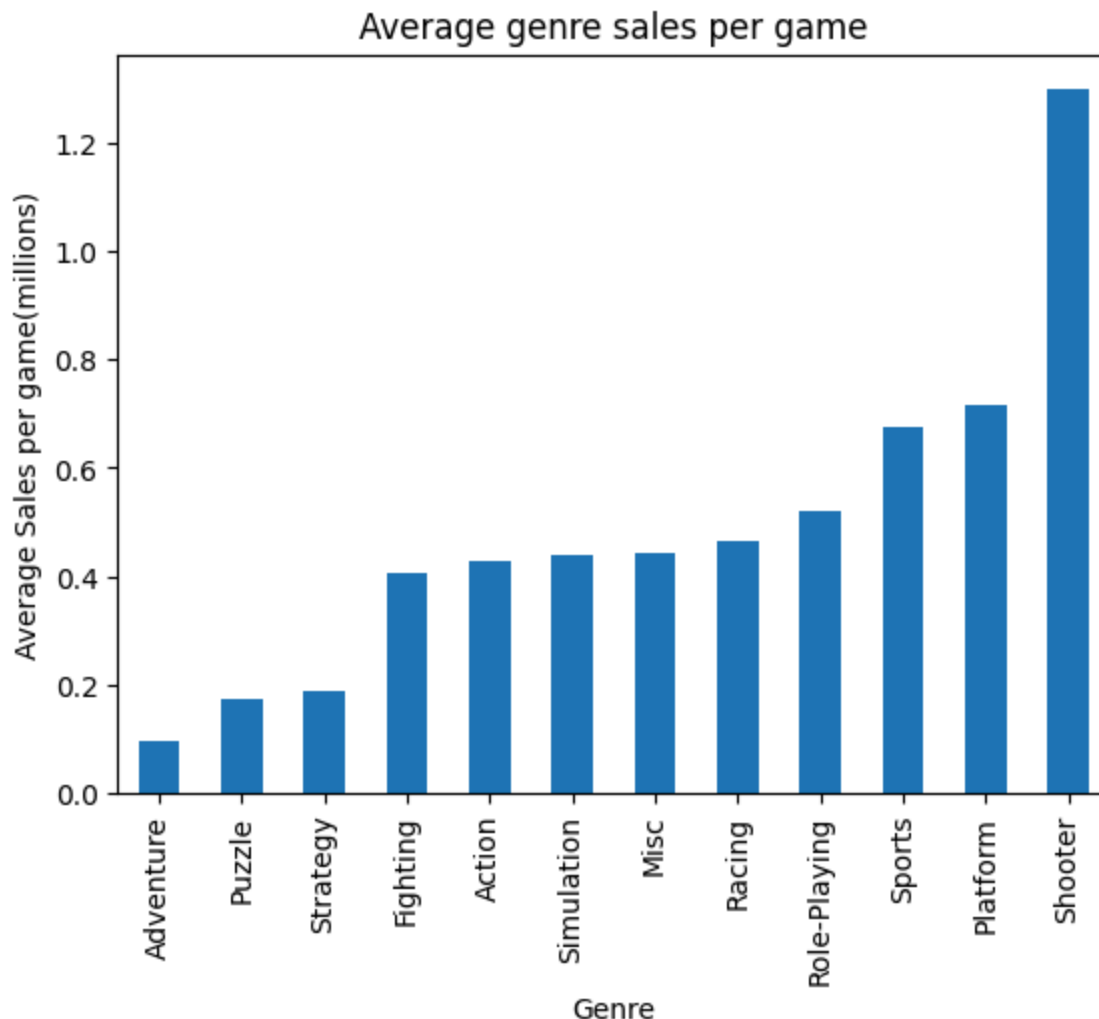
```
genre
Action      441.12
Shooter     304.73
Role-Playing 192.80
Sports      181.07
Misc        85.04
Platform     61.00
Racing       53.50
Fighting     44.49
Simulation   35.12
Adventure    29.43
Strategy     13.34
Puzzle        4.89
Name: total_sales, dtype: float64
```

```
In [33]: genre_list = games_relevant_years['genre'].unique()
print(genre_list)
type(genre_list)
```

```
['Action' 'Shooter' 'Role-Playing' 'Platform' 'Misc' 'Simulation' 'Sports'
 'Fighting' 'Racing' 'Strategy' 'Adventure' 'Puzzle']
```

```
Out[33]: numpy.ndarray
```

```
In [34]: games_relevant_years.groupby('genre')['total_sales'].mean().sort_values().plot(kind
plt.xlabel('Genre')
plt.ylabel('Average Sales per game(millions)')
plt.title('Average genre sales per game')
plt.show()
```



Although "Action" genre was the one with more sales, it isn't the one if we calculate the average sales per game, resulting on a bit more than 0.4 millions per game. On the other hand, the genre "Shooter" (The second on sales), has the highest average sales per game, averaging about 1.35 millions per game.

```
In [35]: #Calculating total sales across the genres
total_market_sales = genre_sales.sum()

#To calculate the share for each genre, we have to:
genre_market_share = (genre_sales/total_market_sales) *100

print("Market Share by Genre (2012-2016):")
print(genre_market_share.round(2))

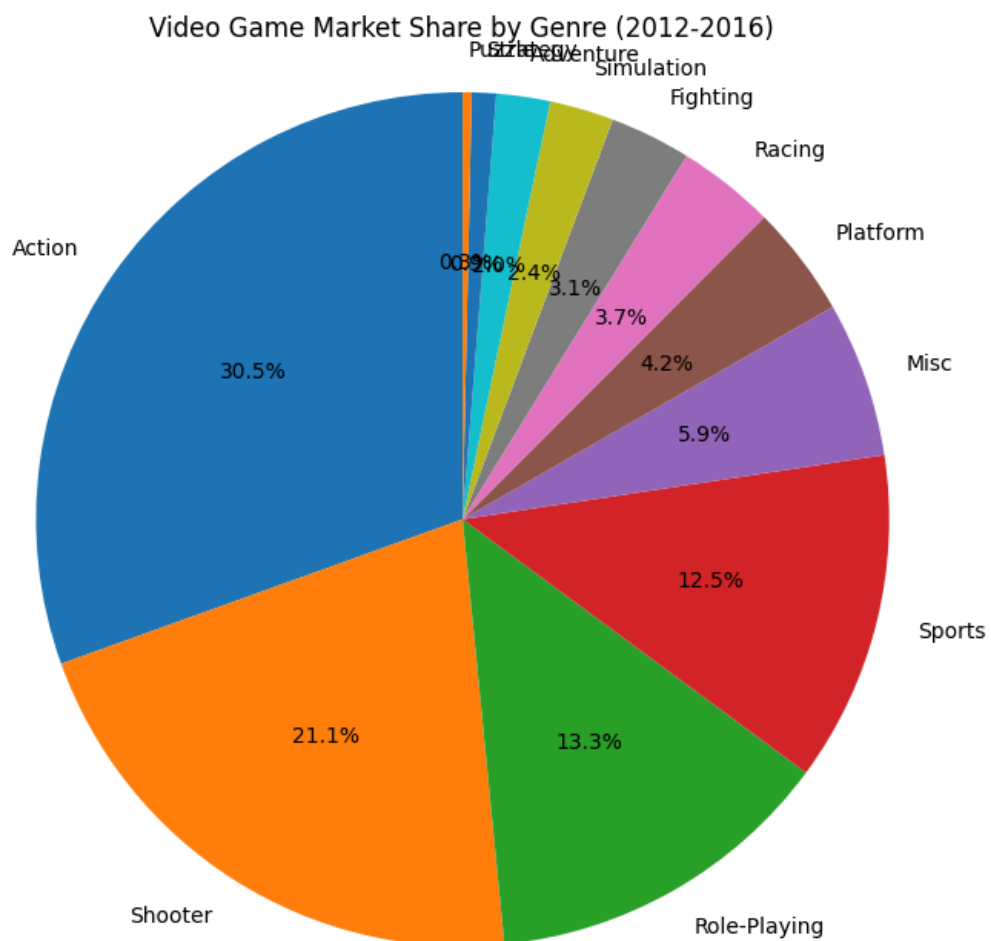
#Calculating the Distribution for the share, we have:
plt.figure(figsize=(10,8))
plt.pie(genre_market_share.values, labels=genre_market_share.index, autopct='%1.1f%%')
plt.title('Video Game Market Share by Genre (2012-2016)')
plt.axis('equal')
```

```
plt.show()
# Calculate market share for each genre
```

Market Share by Genre (2012-2016):

genre	
Action	30.50
Shooter	21.07
Role-Playing	13.33
Sports	12.52
Misc	5.88
Platform	4.22
Racing	3.70
Fighting	3.08
Simulation	2.43
Adventure	2.03
Strategy	0.92
Puzzle	0.34

Name: total_sales, dtype: float64



In [36]: `print(genre_sales)`

```

genre
Action      441.12
Shooter     304.73
Role-Playing 192.80
Sports      181.07
Misc        85.04
Platform    61.00
Racing      53.50
Fighting    44.49
Simulation  35.12
Adventure   29.43
Strategy    13.34
Puzzle       4.89
Name: total_sales, dtype: float64

```

Key Questions for Genre Analysis:

- Which genres consistently perform well?
- Are there any genres showing recent growth or decline?
- How does the average performance vary across genres?

After the analyses, "Action" and "Shooter" are the genres with the best performance. However, "Role-Playing" and "Sports" are growing on the time period chosen. The average varies analyzing the genres, there is a discrepancy analyzing the "Action" genre, for it has the highest sales but it is only the 8th by the average sales per game. "Shooter," "Platform," and "Sports" complete the top 3 highest average sales per game.

Step 4: Regional Market Analysis and User Profiles

In this section, we will analyze the gaming market characteristics across three major regions: North America (NA), Europe (EU), and Japan (JP). Our analysis will focus on platform preferences, genre popularity, and the impact of ESRB ratings in each region.

4.1 Regional Platform Analysis

Let's begin by examining platform performance across different regions:

```
In [37]: platform_performance_by_region = games_relevant_years.groupby('platform')[['na_sales', 'eu_sales', 'jp_sales']]
print(platform_performance_by_region.head(5))
```

	na_sales	eu_sales	jp_sales
platform			
X360	140.05	74.52	1.57
PS4	108.74	141.09	15.96
PS3	103.38	106.86	35.29
XOne	93.12	51.59	0.34
3DS	55.31	42.64	87.79

I have tried to run top 5 sales platforms for all three region on the same cell, but I could not do it. So, I ran it by top 5 sales platform from North America to have an comparing analysis.

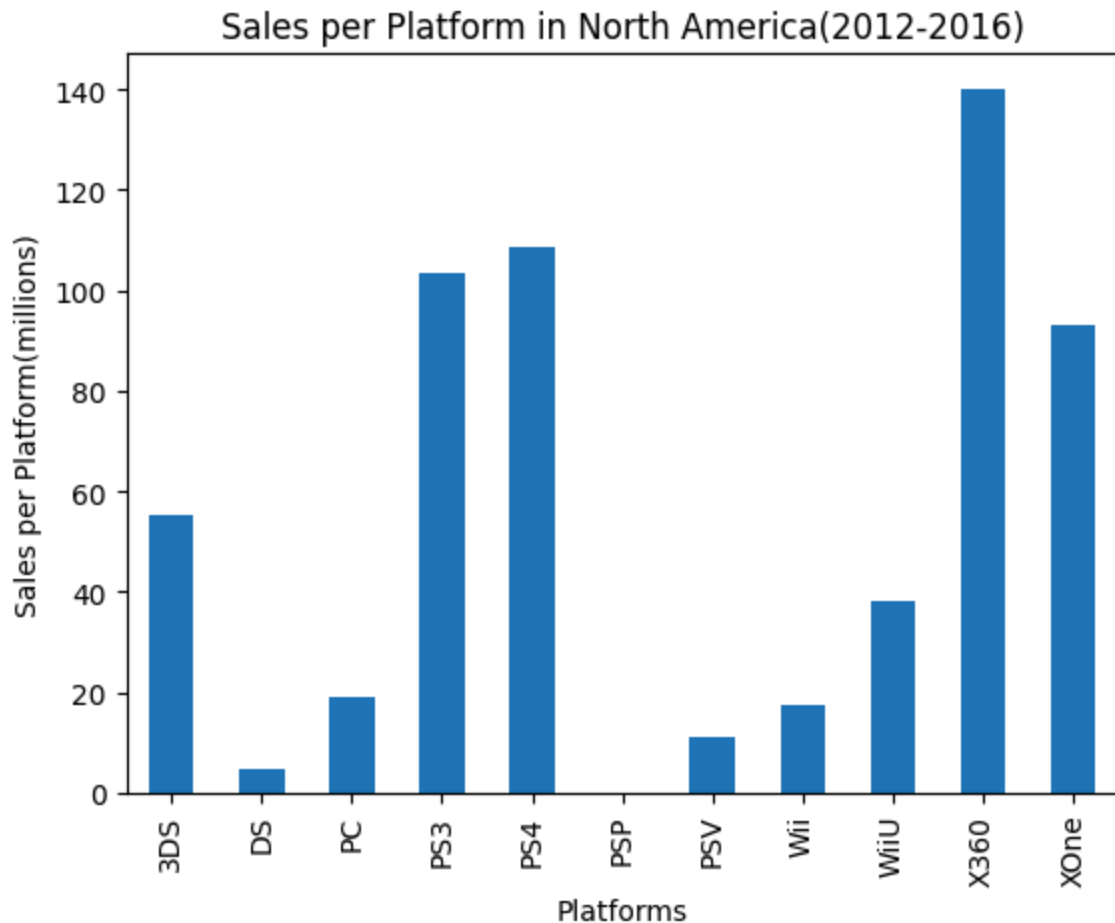
```
In [38]: platform_performance_for_na_sales = games_relevant_years.groupby('platform')['na_sales'].sort_values(ascending=False).head(5)

print(platform_performance_for_na_sales)

#Now, to have a better analysis, let's plot a distribution:
platform_performance_for_na_sales = games_relevant_years.groupby('platform')['na_sales'].sort_values(ascending=False).head(5)
plt.xlabel('Platforms')
plt.ylabel('Sales per Platform(millions)')
plt.title('Sales per Platform in North America(2012-2016)')
plt.show()
# Function to analyze platform performance by region
```

platform	na_sales
X360	140.05
PS4	108.74
PS3	103.38
XOne	93.12
3DS	55.31

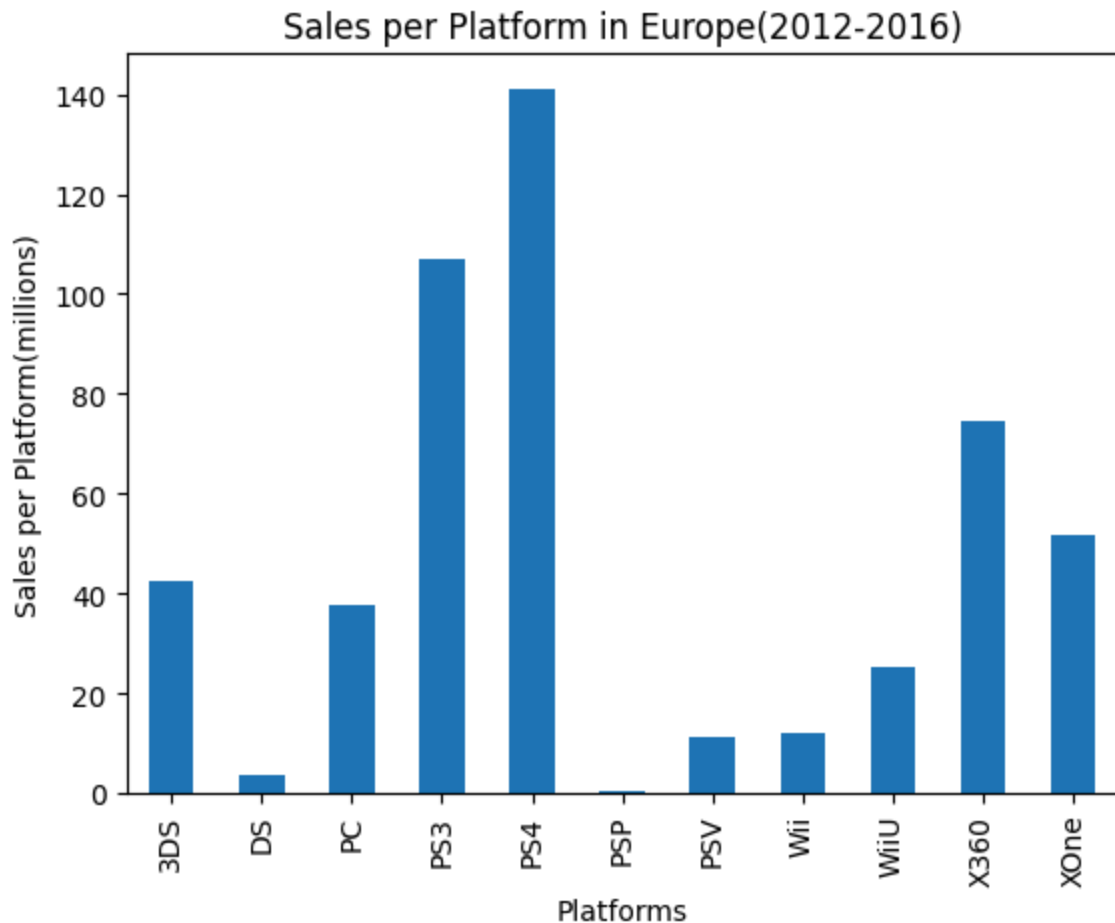
Name: na_sales, dtype: float64



```
In [39]: platform_performance_for_eu_sales = games_relevant_years.groupby('platform')['eu_sales']
print(platform_performance_for_eu_sales.sort_values(ascending=False).head(5))

#Now, to have a better analysis, let's plot a distribution:
platform_performance_for_eu_sales = games_relevant_years.groupby('platform')['eu_sales']
plt.xlabel('Platforms')
plt.ylabel('Sales per Platform(millions)')
plt.title('Sales per Platform in Europe(2012-2016)')
plt.show()
```

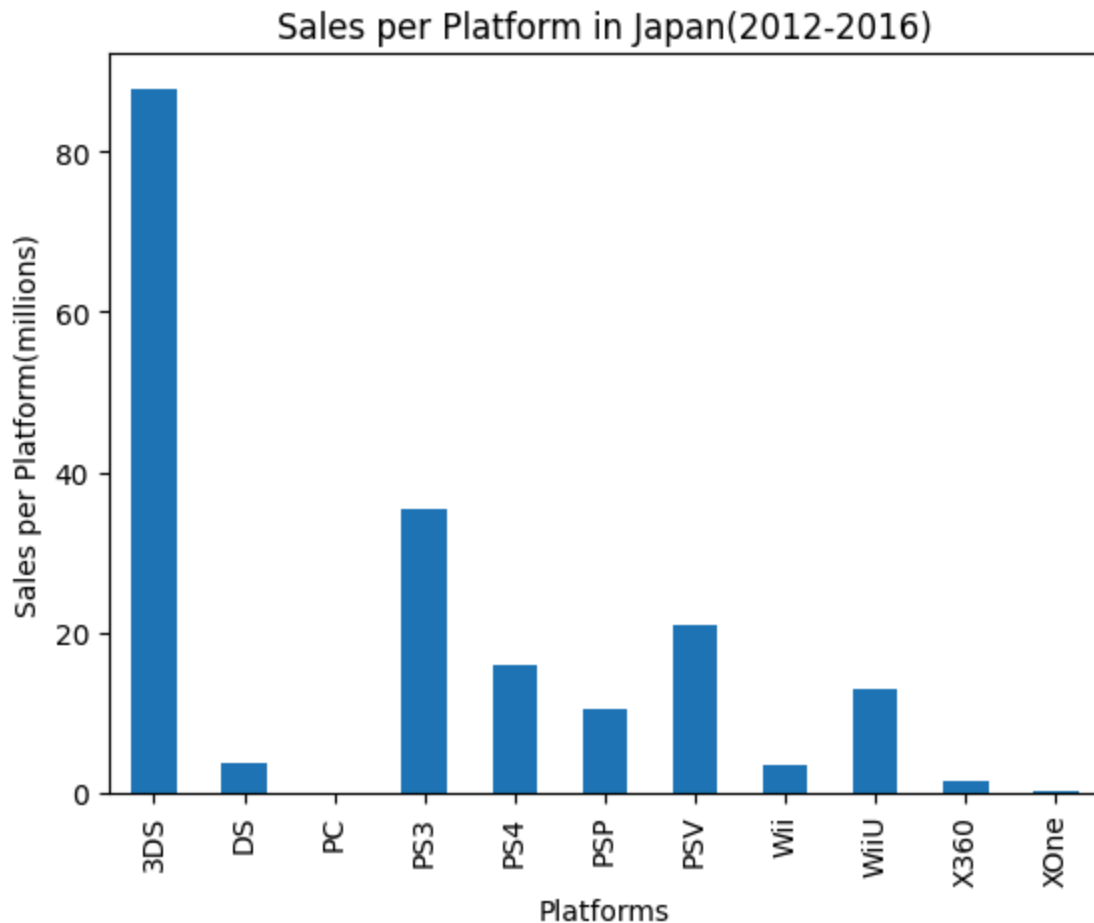
```
platform
PS4      141.09
PS3      106.86
X360      74.52
XOne      51.59
3DS       42.64
Name: eu_sales, dtype: float64
```

```
In [40]: platform_performance_for_jp_sales = games_relevant_years.groupby('platform')['jp_sales']
print(platform_performance_for_jp_sales.sort_values(ascending=False).head(5))

#Now, to have a better analysis, let's plot a distribution:
platform_performance_for_jp_sales = games_relevant_years.groupby('platform')['jp_sales']
plt.xlabel('Platforms')
plt.ylabel('Sales per Platform(millions)')
plt.title('Sales per Platform in Japan(2012-2016)')
plt.show()
```

```
platform
3DS      87.79
PS3      35.29
PSV      21.04
PS4      15.96
WiiU     13.01
Name: jp_sales, dtype: float64
```



After analyzing the regions, we can tell the differences on sales when comparing the three of them. While in North America 'X360' was the biggest on sales, in Europe was 'PS4' and in Japan '3DS'.

Cross-Regional Platform Comparison

Let's create a comparative analysis of platform performance across regions:

```
In [41]: platform_performance_by_region = games_relevant_years.groupby('platform')[['na_sales', 'eu_sales', 'jp_sales']]
print(platform_performance_by_region.head(5))

platform_performance_by_region = games_relevant_years.groupby('platform')[['na_sales', 'eu_sales', 'jp_sales']]
print(platform_performance_by_region.head(5))

platform_performance_by_region = games_relevant_years.groupby('platform')[['na_sales', 'eu_sales', 'jp_sales']]
print(platform_performance_by_region.head(5)) # Create a comparative platform analysis
```

	na_sales	eu_sales	jp_sales
platform			
X360	140.05	74.52	1.57
PS4	108.74	141.09	15.96
PS3	103.38	106.86	35.29
XOne	93.12	51.59	0.34
3DS	55.31	42.64	87.79

	na_sales	eu_sales	jp_sales
platform			
PS4	108.74	141.09	15.96
PS3	103.38	106.86	35.29
X360	140.05	74.52	1.57
XOne	93.12	51.59	0.34
3DS	55.31	42.64	87.79

	na_sales	eu_sales	jp_sales
platform			
3DS	55.31	42.64	87.79
PS3	103.38	106.86	35.29
PSV	10.98	11.36	21.04
PS4	108.74	141.09	15.96
WiiU	38.10	25.13	13.01

I have displayed the top 5 platforms by sales for each of the three regions and compared their sales.

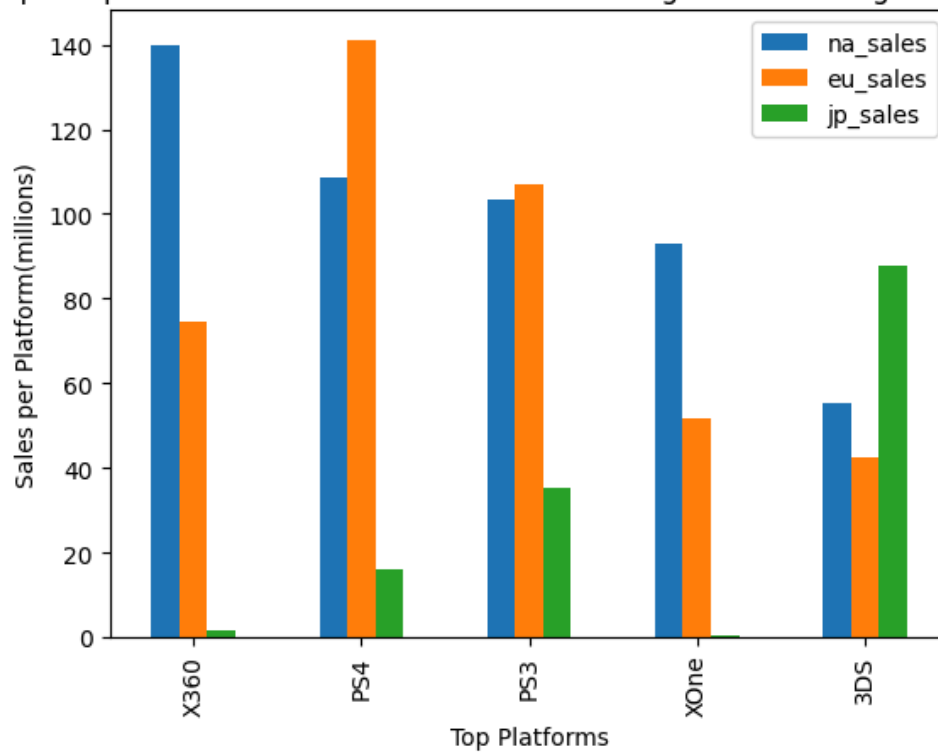
```
In [42]: platform_performance_by_region = games_relevant_years.groupby('platform')[['na_sale
plt.xlabel('Top Platforms')
plt.ylabel('Sales per Platform(millions)')
plt.title('Sales per top 5 Platforms from North America through the three regions(2
plt.show()

platform_performance_by_region = games_relevant_years.groupby('platform')[['na_sale
plt.xlabel('Platforms')
plt.ylabel('Sales per Platform(millions)')
plt.title('Sales per top 5 Platforms from Europe through the three regions(2012-201
plt.show()

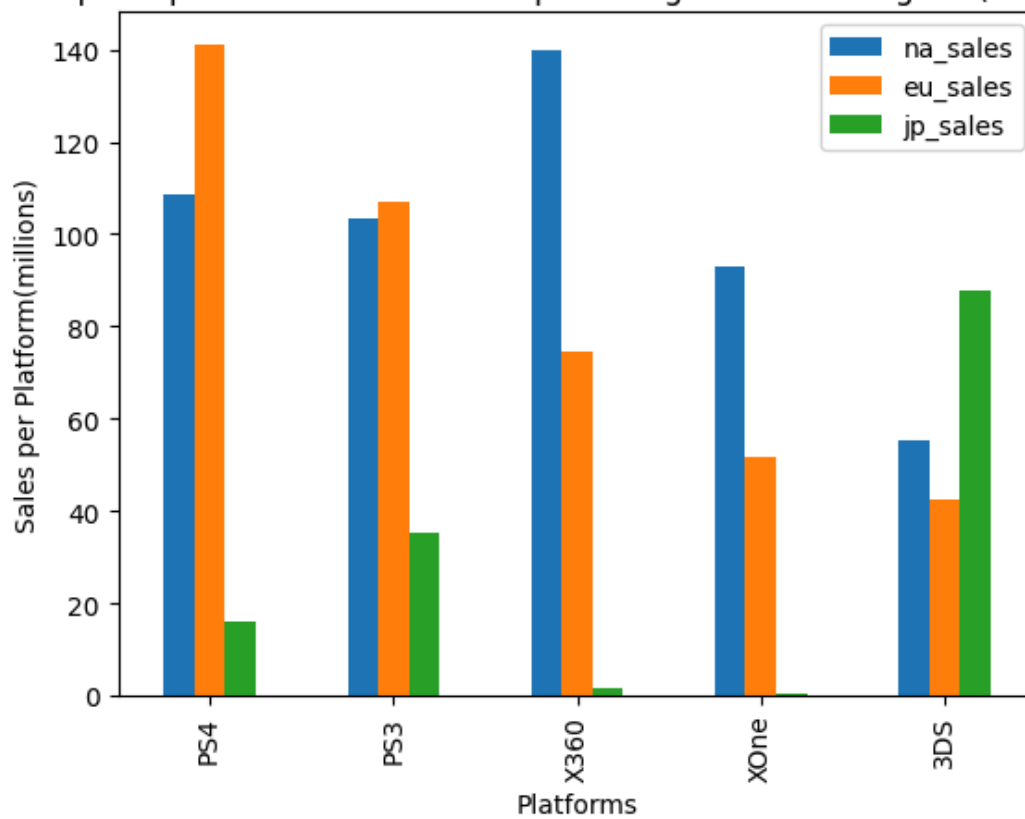
platform_performance_by_region = games_relevant_years.groupby('platform')[['na_sale
plt.xlabel('Platforms')
plt.ylabel('Sales per Platform(millions)')
plt.title('Sales per top 5 Platforms from Japan through the three regions(2012-2016
plt.show()

# Visualize cross-regional comparison for top platforms
```

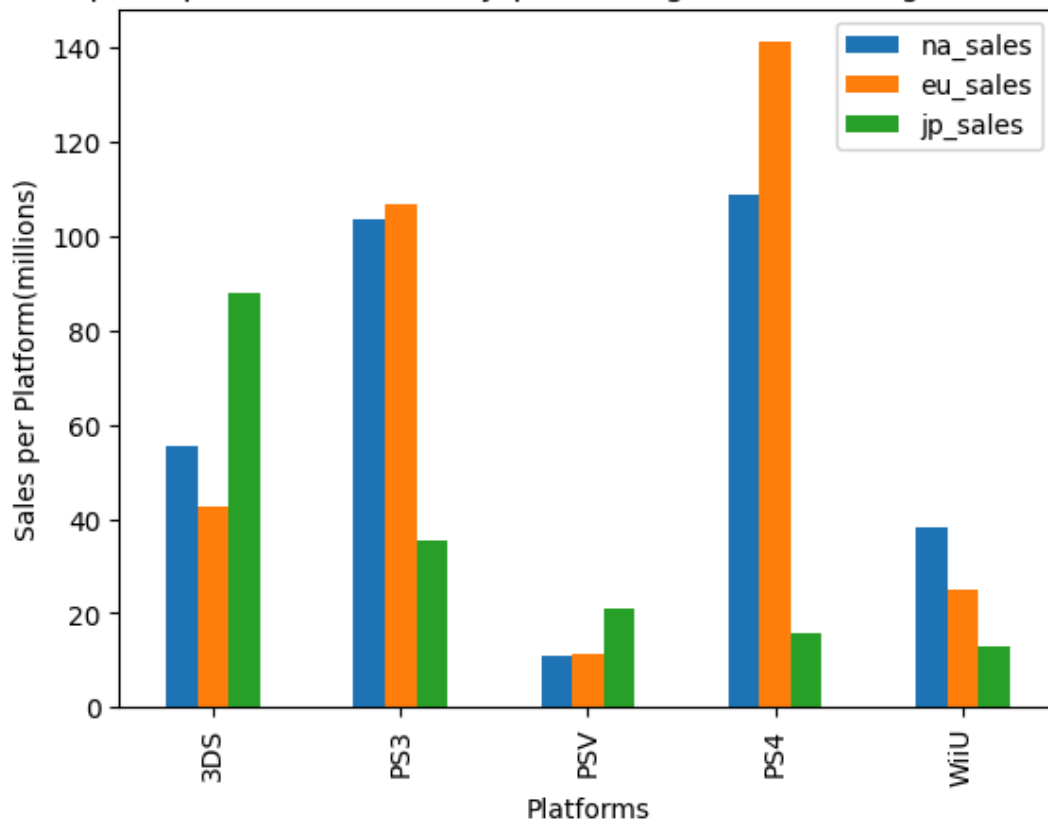
Sales per top 5 Platforms from North America through the three regions(2012-2016)



Sales per top 5 Platforms from Europe through the three regions(2012-2016)



Sales per top 5 Platforms from Japan through the three regions(2012-2016)



I plotted the top 5 platforms by sales for each region, allowing a direct comparison between the three regions.

4.2 Regional Genre Analysis

Now let's examine genre preferences across regions:

```
In [43]: genre_performance_for_na_sales = games_relevant_years.groupby('genre')['na_sales'].
print("Top 5 genre in North America")
print(genre_performance_for_na_sales.head(5))

genre_performance_for_na_sales = games_relevant_years.groupby('genre')['na_sales'].
plt.xlabel('Genres')
plt.ylabel('Sales per Genre(millions)')
plt.title('Sales per Genre in North America(2012-2016)')
plt.show()

# Function to analyze genre performance by region
```

Top 5 genre in Noth America

genre

Action 177.84

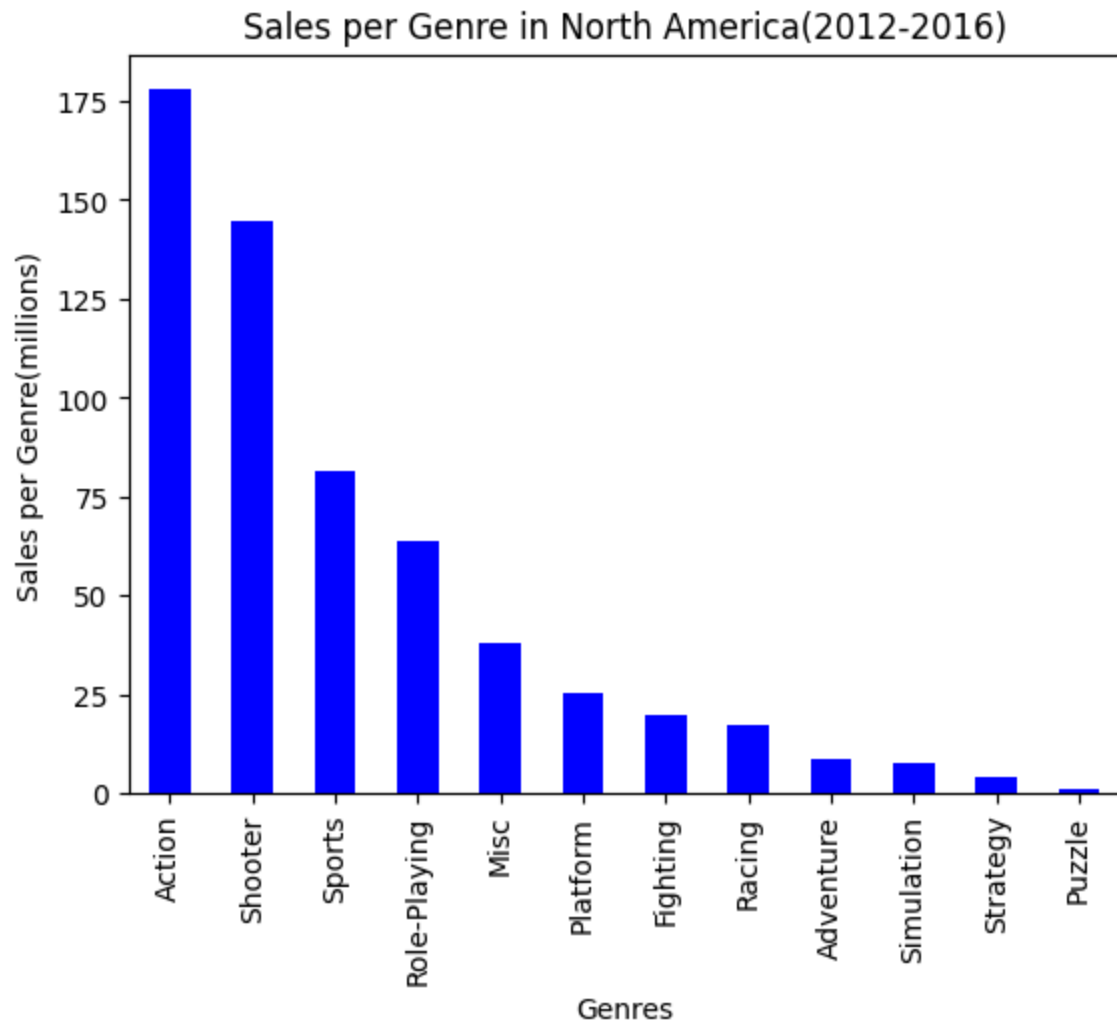
Shooter 144.77

Sports 81.53

Role-Playing 64.00

Misc 38.19

Name: na_sales, dtype: float64



```
In [44]: genre_performance_for_eu_sales = games_relevant_years.groupby('genre')['eu_sales'].
print("Top 5 genre in Europe")
print(genre_performance_for_eu_sales.head(5))

genre_performance_for_eu_sales = games_relevant_years.groupby('genre')['na_sales'].
plt.xlabel('Genres')
plt.ylabel('Sales per Genre(millions)')
plt.title('Sales per Genre in Europe(2012-2016)')
plt.show()
```

Top 5 genre in Europe

genre

Action 159.34

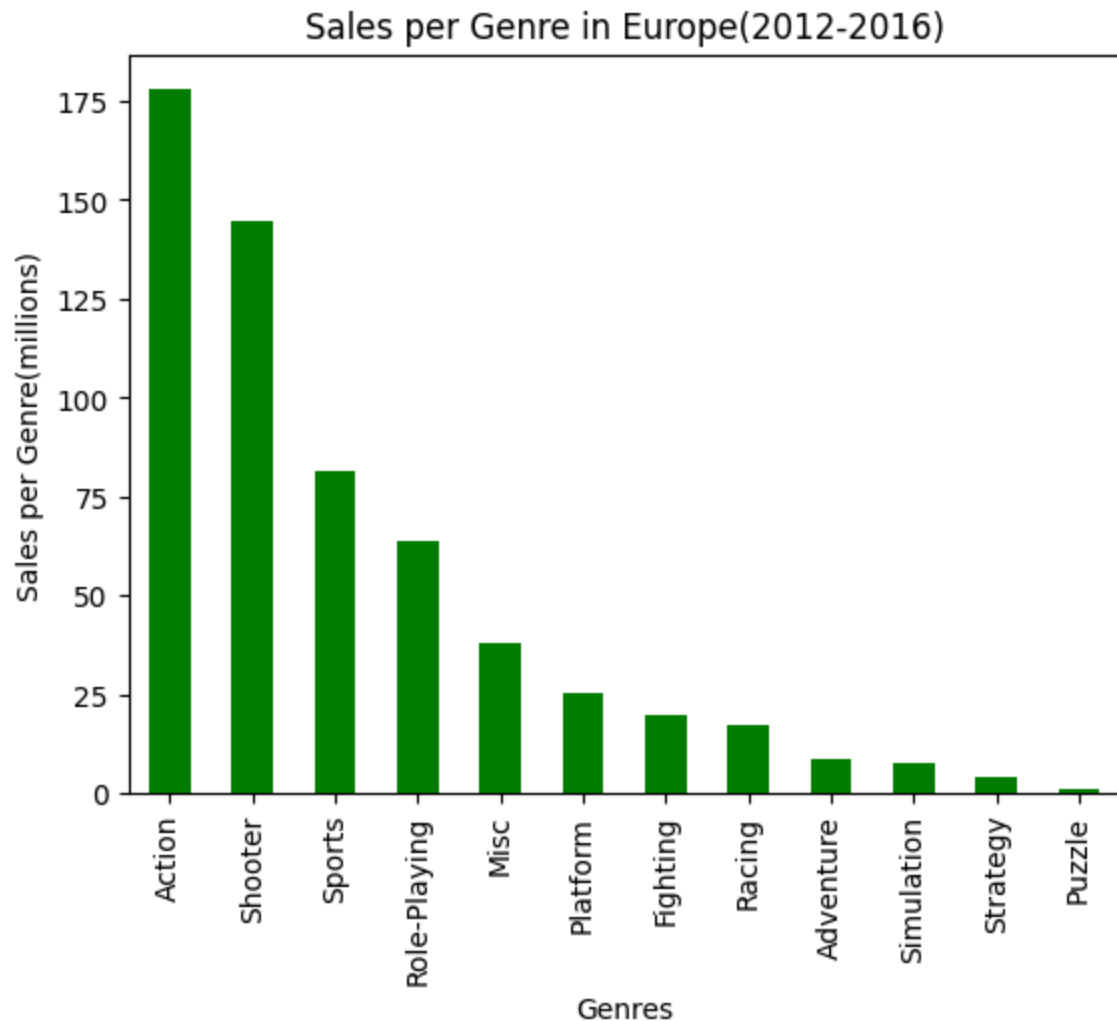
Shooter 113.47

Sports 69.09

Role-Playing 48.53

Racing 27.29

Name: eu_sales, dtype: float64



```
In [45]: genre_performance_for_jp_sales = games_relevant_years.groupby('genre')['jp_sales'].
print("Top 5 genre in Europe")
print(genre_performance_for_jp_sales.head(5))

genre_performance_for_jp_sales = games_relevant_years.groupby('genre')['jp_sales'].
plt.xlabel('Genres')
plt.ylabel('Sales per Genre(millions)')
plt.title('Sales per Genre in Japan(2012-2016)')
plt.show()
```

Top 5 genre in Europe

genre

Role-Playing 65.44

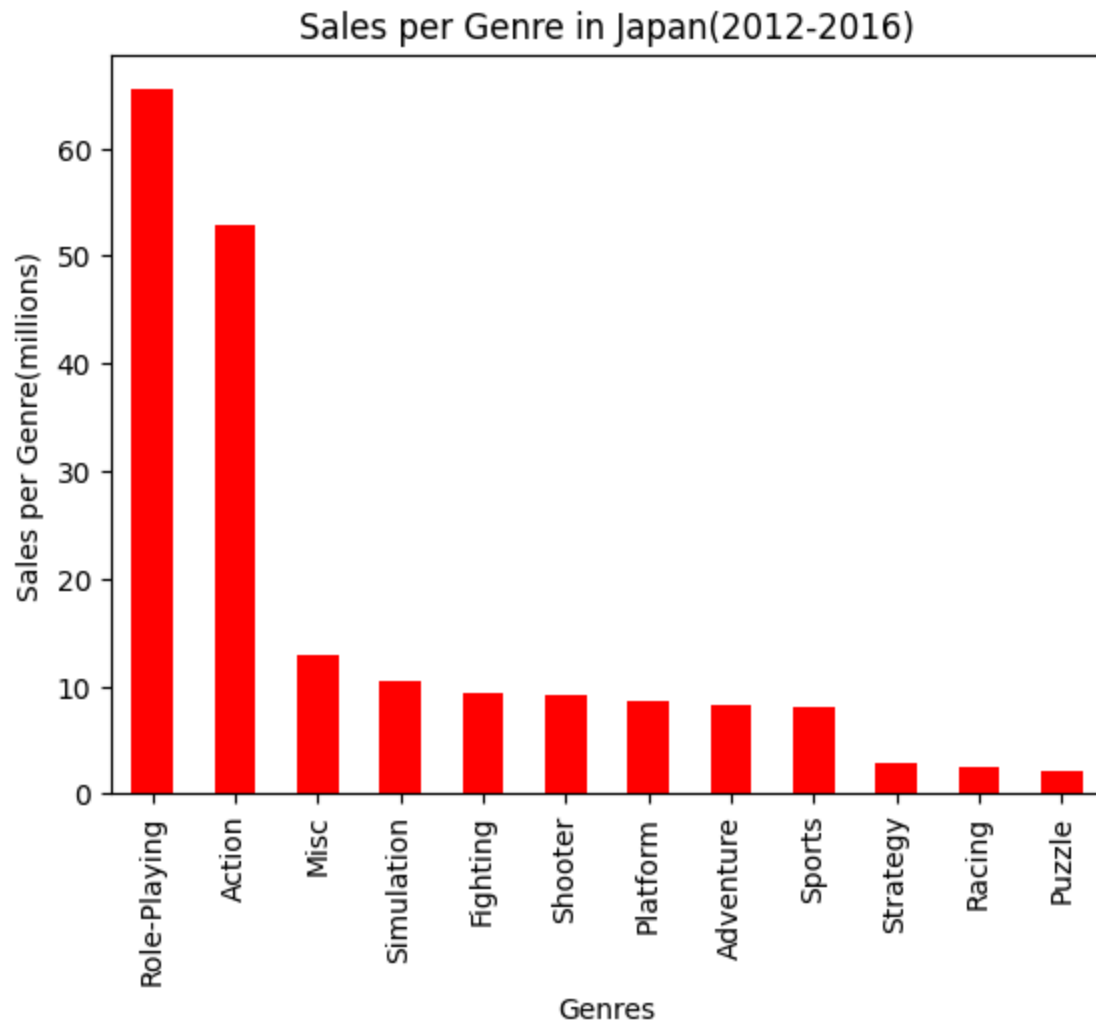
Action 52.80

Misc 12.86

Simulation 10.41

Fighting 9.44

Name: jp_sales, dtype: float64



Cross-Regional Genre Comparison

Let's compare genre preferences across regions:

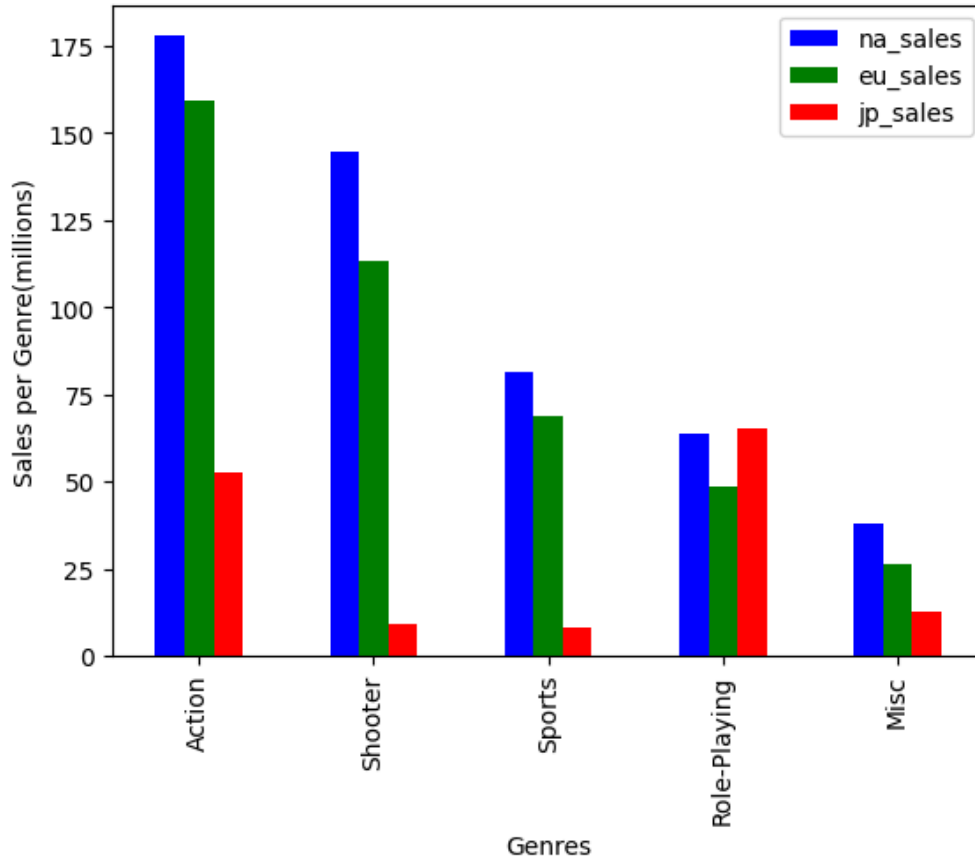
```
In [46]: genre_performance_by_region = games_relevant_years.groupby('genre')[['na_sales', 'e
print(genre_performance_by_region.head(5))

genre_performance_by_region = games_relevant_years.groupby('genre')[['na_sales', 'e

plt.xlabel('Genres')
plt.ylabel('Sales per Genre(millions)')
plt.title('Top 5 Sales per Genre from North America through the three regions(2012-
plt.show()
# Create a comparative genre analysis
```


	na_sales	eu_sales	jp_sales
genre			
Action	177.84	159.34	52.80
Shooter	144.77	113.47	9.23
Sports	81.53	69.09	8.01
Role-Playing	64.00	48.53	65.44
Misc	38.19	26.32	12.86

Top 5 Sales per Genre from North America through the three regions(2012-2016)



I have displayed the top 5 sales per genre from North America, comparing with Europe and Japan so we could have a better understanding about the differences about them. While North America and Europe have both "Action" and "Shooter" as the top 2 genres, Japan has "Role-Playing" as the top one on sales.

4.3 ESRB Rating Impact Analysis

Finally, let's examine how ESRB ratings affect sales in each region:

```
In [47]: rating_performance_for_na_sales = games_relevant_years.groupby('rating')['na_sales']
print("Top 5 rating in North America")
print(rating_performance_for_na_sales.head(5))# Function to analyze ESRB rating imp
```

```

Top 5 rating in Noth America
rating
M          231.57
E          114.37
missing    103.31
E10+       75.70
T           66.02
Name: na_sales, dtype: float64

```

```

In [48]: rating_performance_for_eu_sales = games_relevant_years.groupby('rating')['eu_sales']
print("Top 5 rating in Europe")
print(rating_performance_for_eu_sales)

```

```

Top 5 rating in Europe
rating
M          193.96
E          113.03
missing     91.50
E10+        55.37
T           52.96
Name: eu_sales, dtype: float64

```

```

In [49]: rating_performance_for_jp_sales = games_relevant_years.groupby('rating')['jp_sales']
print("Top 5 rating in Japan")
print(rating_performance_for_na_sales.head(5))

```

```

Top 5 rating in Japan
rating
M          231.57
E          114.37
missing    103.31
E10+       75.70
T           66.02
Name: na_sales, dtype: float64

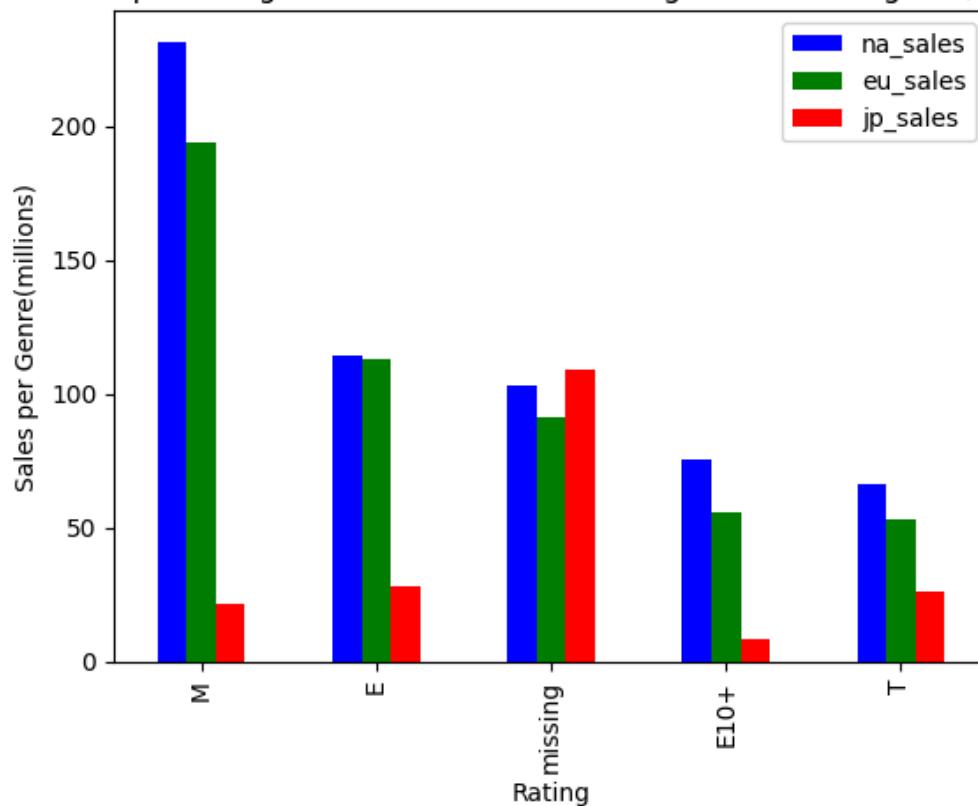
```

```

In [50]: rating_performance_by_region = games_relevant_years.groupby('rating')[['na_sales',
plt.xlabel('Rating')
plt.ylabel('Sales per Genre(millions)')
plt.title('Top 5 Sales per rating from North America through the three regions(2012
plt.show())# Analyze ESRB impact for each region

```

Top 5 Sales per rating from North America through the three regions(2012-2016)



As we can see in the plotting above, North America and Europe have the same top ratings with "M"= Mature(+17) category being the top one and "T"= Teen the last one. However, Japan has E10+(Everyone 10 and older) being the last one.

Step 5 : Hypothesis Tests

—Average user ratings of the Xbox One and PC platforms are the same.

—Average user ratings for the Action and Sports genres are different.

Set the *alpha* threshold value yourself.

Explain:

—How you formulated the null and alternative hypotheses

—What criteria you used to test the hypotheses; and why

Null Hypothesis(H_0) = The Average user ratings of the Xbox One and PC do not differ

Alternative Hypothesis(Hi) = The average user ratings of the Xbox One and PC differs.

Alpha = 0.05, I chose because it is the standard

```
In [ ]: # Get the actual user score arrays to be able to test it.
xbox_one_scores = games_relevant_years[games_relevant_years['platform'] == 'XOne']
pc_scores = games_relevant_years[games_relevant_years['platform'] == 'PC']['user_score']

#Testing the hypothesis:
alpha = 0.05

results = st.ttest_ind(xbox_one_scores, pc_scores, equal_var = False)

print('p-value:', results.pvalue)

if results.pvalue < alpha:
    print("We reject the null hypothesis")
else:
    print("We can't reject the null hypothesis")
```

Because the p-value is way greater than our parameter alpha, we can easily say "We can't reject the null hypothesis". On other words, the average user ratings of the Xbox One and PC, indeed differ.

I have chosen the null hypothesis as the average not differentiating between them. The alternative is the opposite. My criteria was simply compare the ratings and see if the p-values would be small enough for us to determine if the differences between them was big or not.

Step 6. Write a general conclusion

After the analysis, it is clear that we should continue increasing our advertising investment in Action, Shooter, and Sports games. Overall, these genres had the highest sales across all regions. The more we focus on these segments, the more profitable our business is likely to be, and this trend is expected to continue into 2017.